UNIVERSITY OF SOUTHAMPTON
Faculty of Engineering and Physical Sciences
School of Electronics and Computer Science

A project report submitted for the award of
BEng Electronic Engineering

Supervisor: Dr Tomasz Kazmierski
Examiner: Professor Nicholas Harris

Design and Implementation of a RISC-V Sparse Matrix Accelerator on FPGA

by Tochi C. Anyanwu

April 25, 2026

UNIVERSITY OF SOUTHAMPTON

ABSTRACT

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES
SCHOOL OF ELECTRONICS AND COMPUTER SCIENCE

A project report submitted for the award of BEng Electronic Engineering

by Tochi C. Anyanwu

Sparse matrix–dense matrix multiplication (SpMM) is a key kernel in graph neural networks, recommender systems, and compressed neural network inference, but it performs poorly on general-purpose processors because of irregular memory access and low arithmetic intensity. This report presents the design, implementation, and evaluation of a PicoRV32 RISC-V system-on-chip with a custom hardware accelerator for compressed sparse row (CSR) SpMM on the Terasic DE1-SoC Cyclone V FPGA. The design was developed in three optimisation stages: a baseline accelerator with tile length ($TN = 8$) parallel MAC lanes across the dense output dimension, a DSP-oriented MAC enhancement that maps each lane to a dedicated Cyclone V multiplier-accumulator block, and a runtime symmetric INT8 packing stage that stores four quantised operands per 32-bit RAM word. A 12-case hardware benchmark suite evaluating matrix size, dense width, sparsity level, and nonzero pattern was executed in firmware on the synthesised SoC, and all cases passed element-by-element correctness checks. The final INT8 design achieved a peak speedup of 56.80× over the quantised PicoRV32 softcore and 11.92× over an ARM Cortex-M0 softcore with a 2.87× cycle reduction over baseline.

Statement of Originality

- I have read and understood the ECS Academic Integrity information and the University's Academic Integrity Guidance for Students.
- I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have used the open-source picoRV32 RISC-V core by YosysHQ as the soft core processor within my system on chip design.
I has also used the Arm cortex M0 from Arm LTD.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell aiofficer@ecs.soton.ac.uk

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr Tomasz Kazmierski, for his continuous guidance, insightful technical feedback, and encouragement throughout this project. His direction shaped the design approach and helped maintain the clarity and rigour of the evaluation methodology. I am also deeply thankful to my family and friends for their constant support and patience, particularly during the intensive integration and debugging phases of the FPGA deployment.

Contents

Chapter 1

Introduction

1.1 Motivation

Sparse matrix–dense matrix multiplication (SpMM) is one of the fundamental computational kernels in modern machine learning and graph-based computing. In graph neural networks (GNNs) it embodies the neighbourhood aggregation step, in which a sparse adjacency matrix is multiplied against a dense feature matrix [1, 2]. In large-scale recommender systems, collaborative filtering relies on sparse user–item interaction matrices whose dimensions can reach hundreds of millions of rows [3]. In scientific simulation it is the inner loop of iterative solvers for finite-element and finite-difference discretisations. The same pattern appears in transformer attention with sparse masks and in the weight-matrix multiply step of pruned and compressed neural networks [4]. Across all of these workloads the common denominator is that the useful arithmetic is only a small fraction of the work that a general-purpose processor actually performs, with most cycles spent chasing indirect indices, handling row boundaries, and issuing scattered memory accesses.

With the rapid rise in demand for AI, there has been a matching rise in demand for better model algorithms, more data, and above all more compute. On the compute side, most AI workloads are now executed on application specific integrated circuits (ASICs), since these have been shown to maximise throughput, energy efficiency, and cost effectiveness for the particular matrix operations that AI models need to run. Some of the most well known commercial examples are the Tensor Processing Unit (TPU) from Google and the Neural Processing Unit (NPU) from Apple. The design of any such ASIC, however, almost always begins with prototyping on

a Field Programmable Gate Array (FPGA), because FPGAs can execute many operations in parallel and are easily reconfigurable, which keeps the cost and turnaround time of architectural exploration both low and fast. An FPGA-hosted accelerator for SpMM is therefore a natural first step towards a fully custom silicon implementation, and is the platform targeted by this project.

1.2 Project Aim

The aim of this project is to design, implement, and evaluate a RISC-V compatible hardware accelerator for compressed sparse row (CSR) SpMM on the Terasic DE1-SoC FPGA board. The accelerator must integrate cleanly into a small system-on-chip (SoC), share on-chip block RAM with the CPU, and deliver a measurable, reproducible speedup over a comparable software implementation while remaining understandable at the register-transfer level (RTL) and transparent in its benchmarking methodology. A secondary aim is to isolate the individual contribution of each optimisation step, so that the dominant performance bottleneck can be identified through direct measurement rather than assumption. To achieve this, the design is developed in three stages (a baseline accelerator, a parallel DSP-oriented multiply-accumulate enhancement, and a symmetric INT8 packing stage), forming a controlled experimental path in which only one design dimension is changed at a time.

Beyond the core aim, the project defined three stretch goals at the outset. The first was to introduce a custom RISC-V instruction to trigger the accelerator directly from the CPU pipeline, rather than through a memory-mapped register write, which would reduce launch overhead and demonstrate that the PicoRV32 softcore can be extended at the instruction set level. The second was to implement runtime INT8 quantisation so that operand fetch bandwidth could be reduced by packing four quantised values into a single 32-bit RAM word. The third was to extend the accelerator with optional activation functions and a max-pooling stage, so that it could serve as a building block for a small inference pipeline rather than as a standalone kernel. Of the three, the INT8 quantisation goal was achieved and is reported in full in this work; the remaining two are discussed as future work in Chapter 8.

1.3 Report Structure

The remainder of this report is structured as follows. Chapter 2 reviews the background needed to justify the design choices, covering SpMM workloads, sparse data formats, prior accelerator work, the rationale for FPGAs, and the INT8 bandwidth argument. Chapter 3 defines the implemented SoC platform and its memory-mapped I/O interface. Chapter 4 presents the accelerator architecture and each optimisation step in detail. Chapter 5 defines the hardware verification approach and the unit and integration testbenches used to check the RTL. Chapter 6 describes the bare-metal firmware, the benchmark suite, and the measured cycle-count and correctness results across the three optimisation stages. Chapter 7 discusses lessons learned and project management, and Chapter 8 closes the report with a summary and proposed future work.

Chapter 2

Background and Literature Review

There is a vast body of algorithms for Sparse Matrix Multiplication (SpMM), as well as many different storage formats for converting sparse matrices into representations that can be operated on efficiently. This chapter provides the information needed to understand the context and design decisions taken throughout this project, alongside the goals that were set out at the start. If any of the material that follows is unfamiliar, a more elementary overview of SpMM and the CSR format is given in Appendix A, and further reading can be pursued through the references cited in each section.

2.1 SpMM in Modern Workloads

The motivation for a dedicated SpMM accelerator is made concrete by the sparsity levels that real workloads exhibit. Figure 2.1 summarises representative density ranges across four workload classes: in each case, the fraction of nonzero entries is small enough that storing and operating on the matrix in dense form would be wasteful by one to five orders of magnitude.

These sparsity patterns appear in several domains, but the hardware problem is similar in each case. GNN layers multiply a sparse adjacency matrix by dense feature rows [1, 2]; recommender systems apply sparse user–item interaction matrices to dense embedding tables [3]; scientific solvers repeatedly apply sparse discretisation matrices to blocks of vectors; and pruned neural networks execute sparse weight

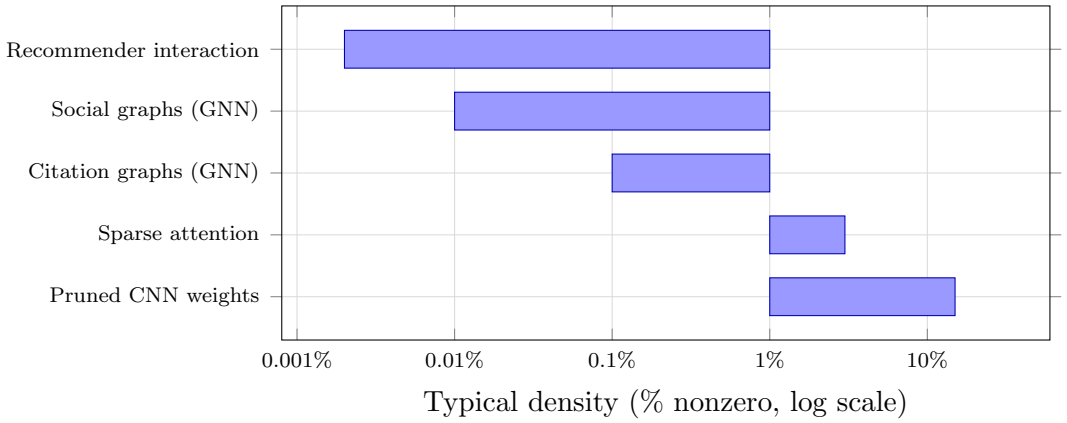

FIGURE 2.1: Representative density ranges for matrices encountered in four classes of modern workload. Social and citation graphs in GNN benchmarks such as `ogbn-arxiv` and `Reddit` typically have densities below 0.1% [1]; large-scale recommender interaction matrices are often two orders of magnitude sparser still [3]. Pruned CNNs sit at the other end of the range, retaining 10–20% of their weights after aggressive pruning [4]. In every case, an SpMM execution model that iterates only over the nonzeros avoids between 90% and $> 99.99\%$ of the arithmetic that a dense multiply would perform.

matrices after most connections have been removed [4]. The common feature is that the useful arithmetic is tied to the nonzero structure of A, while the dense operand B still has to be streamed and accumulated efficiently.

2.2 Why Sparsity Changes the Design Problem

2.2.1 The FLOP-to-Bandwidth Mismatch

Dense matrix multiplication is efficient because it has high operational intensity: for an $M \times K \times N$ General Matrix Multiply (GEMM), many Floating Point Operations (FLOPs) are performed for every byte of data loaded from memory, with each operand reused across the N output columns. Sparse matrix multiplication breaks this reuse pattern. For a CSR SpMM with NNZ nonzeros, the number of useful MACs per byte of CSR index data fetched is

$$\mathrm{OI} = \frac{2 \cdot \mathrm{NNZ} \cdot N}{\mathrm{NNZ} \cdot (4+4) + (M+1) \cdot 4 + \mathrm{NNZ} \cdot 4 \cdot N},$$

where the denominator accounts for `colidx` (4 bytes per entry), `values` (4 bytes per entry), `rowptr` (4 bytes per pointer), and the dense B segment reads (4 bytes per column per nonzero). For small N and low density, the operational intensity is

significantly below the roofline knee of typical on-chip BRAM systems [5], indicating that SpMM is memory-bandwidth-limited rather than compute-limited on most platforms.

2.2.2 Irregular Access and Control-Flow Overhead

Beyond raw bandwidth, SpMM imposes control overhead that dense accelerators are not designed to handle. For each nonzero a_{ij}, the hardware must:

1. read the column index j from `colidx`;
2. compute the address of the relevant row of B as B_base $+ j \cdot N$;
3. read the N values of $B[j, :]$; and
4. accumulate $a_{ij} \cdot B[j, n]$ into $C[i, n]$ for each column n.

Steps (1) and (2) introduce indirection, often called "pointer-chasing": the next address depends on a value that has just been read. This breaks prefetching, and on an in-order core such as PicoRV32 each stall is exposed directly in the cycle count, so control overhead rather than arithmetic becomes the primary bottleneck. Figure 2.2 contrasts the predictable stride of a dense traversal with the indirect, data-dependent address sequence that CSR SpMM produces.

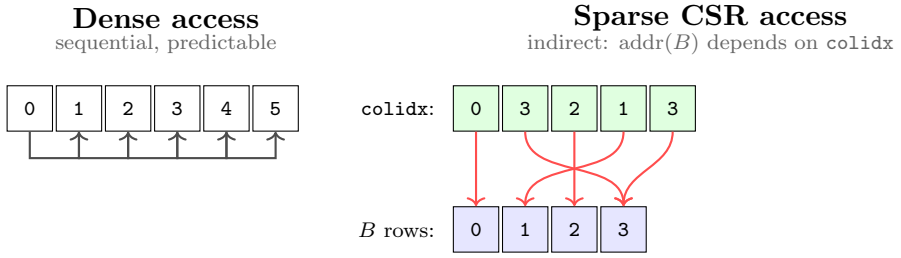

FIGURE 2.2: Memory access patterns in dense versus sparse CSR traversal. Dense access strides sequentially through contiguous addresses, so a prefetcher can fetch ahead. CSR access loads a column index from `colidx` first, then uses that value as an indirect pointer into B; the next address cannot be known until the previous load completes, which breaks prefetching and produces data-dependent cache misses.

2.3 Sparse Matrix Storage Formats

Several sparse matrix storage formats have been proposed; the choice of format strongly affects the hardware control structure. Table 2.1 summarises the most relevant formats and their suitability for hardware acceleration.

Format	Acronym	Description	Hardware suitability
Coordinate list	COO	Stores (row, col, val) triplets unsorted or sorted by row. Simple to generate; requires sorting for efficient row-wise access.	High overhead per nonzero; poor for row-streaming hardware.
Compressed Sparse Row	CSR	Three arrays: `rowptr` ($M+1$ entries), `colidx` (NNZ), `values` (NNZ). Row bounds are contiguous.	Excellent for row-wise traversal; natural hardware FSM mapping.
ELLPACK	ELL	Pads each row to the maximum NNZ per row; stores in column-major order.	Good for regular sparsity; wastes storage and bandwidth when rows vary widely.
Block Compressed Sparse Row	BSR	Groups nonzeros into dense $r \times c$ blocks stored in CSR form.	High throughput on block-sparse matrices; unsuitable for unstructured sparsity.
Compressed Sparse Column	CSC	Transpose of CSR; columns are compressed.	Natural for column-wise access patterns; awkward for row-oriented output.

TABLE 2.1: Comparison of common sparse matrix storage formats and their suitability for hardware acceleration.

2.3.1 Why CSR Was Chosen

CSR was selected because it matches the row-wise execution order used by both the firmware baseline and the accelerator. The `rowptr` array gives constant-time access to the start and end of each output row, while `colidx` and `values` can then be streamed sequentially. This is the layout assumed by Gustavson-style sparse multiplication [6] and remains the default CPU/GPU representation for row-oriented SpMV and SpMM [7]. For a small hardware FSM it avoids COO sorting,

ELL padding, BSR block addressing, and column-oriented output reconstruction. A worked CSR encoding example is provided in Appendix A.2.

2.4 SpMM Execution with CSR

The previous section explained how CSR encodes a sparse matrix. This section explains how a sparse–dense matrix multiply actually uses that encoding. The distinction matters for hardware, because the same three arrays that describe the storage of A also drive the address generation of the accelerator.

2.4.1 Problem Statement

Given a sparse matrix $A \in \mathbb{R}^{M\times K}$, a dense matrix $B \in \mathbb{R}^{K\times N}$, and an output $C \in \mathbb{R}^{M\times N}$, the product $C = AB$ is defined element-wise by

$$C[i,n] \;=\; \sum_{j=0}^{K-1} A[i,j]\cdot B[j,n].$$

For a dense A the inner sum performs K multiplies per output element, the vast majority of which are zero when A is sparse. CSR removes those zeros from the loop entirely: only the nonzero entries of each row of A are enumerated, and each one contributes a scaled copy of a single row of B to the corresponding row of C.

In CSR form, the computation iterates over each output row and then over the nonzeros in that row:

1. For each output row $i = 0, 1, \ldots, M-1$, look up the row bounds $p_{\text{start}} = \texttt{rowptr}[i]$ and $p_{\text{end}} = \texttt{rowptr}[i+1]$.
2. For each $p \in [p_{\text{start}}, p_{\text{end}})$, read the column $j = \texttt{colidx}[p]$ and the scalar $a = \texttt{values}[p]$, fetch the dense row $B[j,:]$, and accumulate $C[i,:] \mathrel{+}= a\cdot B[j,:]$.

The inner update is a scalar-times-vector operation across the N output columns: each nonzero of A scales one row of B and accumulates that row into $C[i,:]$. This is why the accelerator parallelises across the dense dimension rather than across the sparse dimension. The control FSM only needs the three CSR arrays for address generation, while the MAC lanes see regular work over a contiguous segment of B. Full pseudocode and a worked numeric example are left to Appendix A.4.

2.5 Prior Work on SpMM Accelerators

Table 2.2 summarises the accelerator work most relevant to this project.

Work	Target	Relevance to this project
OuterSPACE [8]	SpGEMM	Outer-product sparse–sparse multiplication with dedicated accumulation.
SpArch [9]	SpGEMM merge phase	Shows that sparse output merging can dominate when both inputs are sparse.
ExTensor [10]	Sparse tensor algebra	Uses intersection hardware to skip absent operand pairs before multiplication.
SIGMA [11]	Sparse DNN training	Reconfigures its reduction network to match unstructured sparse weights.

TABLE 2.2: Representative sparse-acceleration designs and their relationship to this work.

These designs target high-throughput sparse–sparse accumulation, tensor intersection, or reconfigurable reduction networks, and assume scheduling machinery or memory systems well beyond a small Cyclone V SoC. This project therefore focuses on the narrower CSR SpMM case: one sparse input, one dense input, a row-wise output order, and a datapath sized around the BRAM and DSP resources available on the DE1-SoC.

2.6 Hardware Platform Alternatives

General-purpose CPUs handle SpMM inefficiently because the irregular access pattern defeats prefetching and out-of-order speculation, and on a small in-order soft-core such as PicoRV32 there is no data cache at all. GPUs achieve high SpMM throughput through massive parallelism [7] but require 100–400 W and a substantial runtime stack, which rules them out for an embedded SoC. A custom ASIC would deliver the best throughput and power, but the development cost and fabrication lead time are impractical for a final-year project.

FPGAs provide a practical middle ground: custom datapath parallelism, flexible on-chip memory organisation, and far lower power than a GPU [12]. The Cyclone V on the DE1-SoC supplies M10K BRAM blocks, DSP multiplier-accumulator blocks, and configurable logic that can be combined into a bespoke CSR SpMM datapath with a memory interface tailored to the access pattern, which is a practical choice for prototyping an accelerator architecture before any ASIC commitment.

2.7 INT8 Quantisation

INT8 is included in this background chapter only because it changes the memory traffic argument: four signed 8-bit operands fit in one 32-bit BRAM word. For a memory-bound CSR SpMM kernel, this attacks the operand-supply bottleneck more directly than widening the MAC array. The implementation uses symmetric linear quantisation [13, 14]: for a vector $\mathbf{v}$, $v_{\max} = \max_i |v_i|$ is computed and each value is mapped as

$$q_i = \mathrm{clip}\left(\mathrm{round}\left(\frac{v_i}{v_{\max}} \cdot 127\right), -127, 127\right), \tag{2.1}$$

with scale $s = v_{\max}/127$. Symmetry removes the zero-point correction that an affine quantiser would add to every MAC, and the hardware accumulates INT8×INT8 products into INT32 rather than back into INT8. The detailed quantisation and dequantisation examples are in Appendix A.6.

2.8 Roofline Analysis and Design Implications

The roofline model [5] characterises whether a kernel is compute-bound (throughput limited by the peak arithmetic rate) or memory-bound (throughput limited by available bandwidth). The position of a kernel on the roofline is determined by its *arithmetic intensity*: the ratio of arithmetic operations to bytes of data movement. A kernel whose intensity places it to the left of the roofline knee is memory-bound, and its achievable throughput scales linearly with bandwidth; one placed to the right is compute-bound, and its throughput is capped by the peak arithmetic rate.

SpMM sits firmly in the memory-bound regime for the reasons developed in Section 2.2: low operand reuse and indirect access keep the arithmetic intensity well below the knee. The practical implication for accelerator design is that widening the MAC array yields only a small gain unless operand supply is widened alongside it. Quantisation addresses this directly by increasing the number of useful operands packed into each memory transaction, moving the design's operating point along the bandwidth roof towards the compute-bound plateau. A numerical instantiation of this argument for the present $TN = 8$ accelerator is given in Chapter 4, and Figure 2.3 illustrates the qualitative shift that INT8 packing produces.

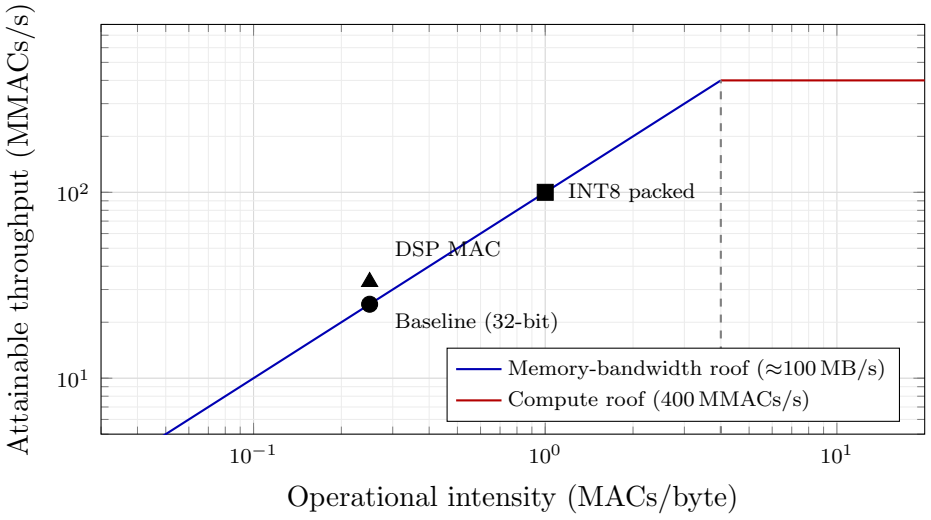

FIGURE 2.3: Roofline interpretation of the three accelerator stages on the Cyclone V platform. The baseline and DSP-enhanced designs sit on the memory-bandwidth roof: arithmetic throughput is capped by operand fetch, so widening the MAC array yields only a small gain. INT8 packing raises the operational intensity by a factor of four (four operands per 32-bit word), shifting the design rightward and upward along the bandwidth roof towards the compute-bound region.

Chapter 3

System-on-Chip Design

This chapter describes the system that hosts the SpMM accelerator: what the SoC looks like as a whole, why a soft RISC-V core was chosen as the host, how memory and the bus are organised, and how firmware drives the accelerator through a small bank of memory-mapped registers. The accelerator itself is treated as a black box here; its internal datapath and optimisation are the subject of Chapter 4.

3.1 System Overview

The SoC is synthesised entirely into FPGA fabric on the Terasic DE1-SoC development board [15]. The hard ARM Cortex-A9 on the HPS side of the device is unused, so the whole design is self-contained in programmable logic. All benchmark code and data reside on chip: firmware, CSR arrays, the dense matrix B, and the output matrix C all reside in a shared 64 KB dual-port block RAM. The entire SoC is clocked from the DE1-SoC's 50 MHz on-board oscillator.

Keeping the design entirely in FPGA fabric removes the HPS, Linux, AXI bridge, and external-memory effects from the measurement path. This makes CPU and accelerator cycle counts directly comparable, but limits benchmark sizes to what fits in the 64 KB shared BRAM. The shared-memory structure is therefore the central platform constraint.

Figure 3.1 shows the top-level block diagram. The PicoRV32 CPU, the SpMM accelerator, UART, and GPIO all hang off a simple interconnect that routes requests to the BRAM or to one of the memory-mapped peripheral regions.

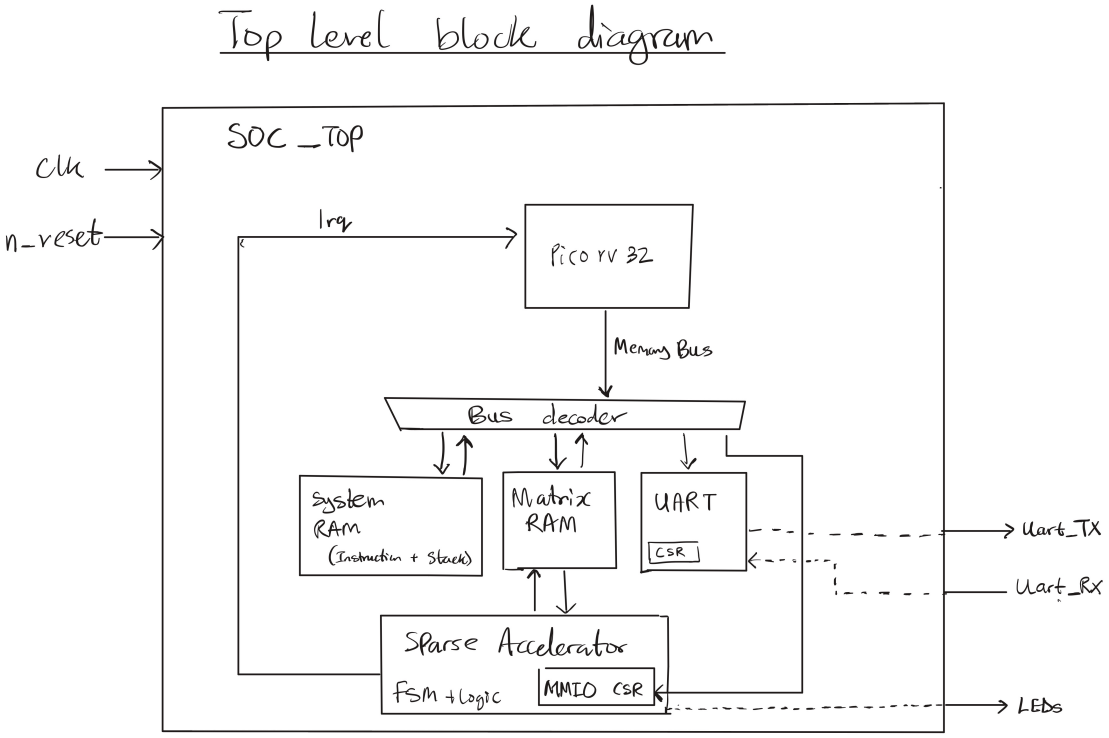

FIGURE 3.1: Top-level RISC-V SoC block diagram.

3.2 Why a Soft RISC-V SoC

The host processor is PicoRV32 [16], a compact RV32IM soft-core implementing the RISC-V base integer instruction set with the hardware integer multiply extension [17]. Three practical reasons drove the choice.

PicoRV32 is small, roughly 700 to 1000 ALMs depending on configuration, which leaves the majority of the Cyclone V fabric available for the accelerator and the 64 KB BRAM. It exposes the `rdcycle` performance counter CSR natively, so all benchmark timing in this report is cycle-accurate without needing an external timer or off-chip instrumentation. And its memory bus is a plain valid/ready handshake with address, data, and write-enable signals, which drops straight onto the SoC interconnect without an AXI adapter.

The main limitation is that PicoRV32 is an in-order, non-pipelined core with no cache, so the software baseline is weaker than a modern superscalar or vector CPU. The comparison is still appropriate because both software and hardware run on the same FPGA SoC and use the same memory system. The M extension is enabled

so the software reference uses hardware multiply rather than an artificially slow software routine.

3.3 Target FPGA Platform

The Cyclone V 5CSEMA5F31C6 on the DE1-SoC provides the resources summarised in Table 3.1 [18].

Resource	Total available	Used (estimated)
Adaptive Logic Modules (ALMs)	32,070	≈5,000 (16%)
Logic registers	128,280	≈8,000 (6%)
M10K memory blocks (10 Kbit each)	397	≈20 (5%)
DSP 18×18 multiplier blocks	87	≈8 (9%)
I/O pins	457	—

TABLE 3.1: Cyclone V 5CSEMA5F31C6 resource availability and estimated design utilisation.

The critical resources are M10K BRAMs and DSP blocks. The shared 64 KB RAM consumes around 16 M10Ks [19], while the eight-lane accelerator maps naturally onto eight DSP multiplier blocks [20]. The estimated utilisation leaves sufficient headroom for the control logic and small local buffers.

3.4 Memory and Interconnect Architecture

3.4.1 Bus Protocol

The SoC interconnect uses the native PicoRV32 valid/ready memory bus with 32-bit address and data signals, a 4-bit byte-enable mask, and a read/write indicator. All signals are synchronous to the 50 MHz system clock. BRAM reads have one registered cycle of latency, while MMIO register accesses are combinatorial.

3.4.2 Dual-Port BRAM Organisation

The 64 KB on-chip RAM is implemented as an Altera `altsyncram` true-dual-port memory. Port A is used by the CPU with byte enables, while Port B is used by the accelerator with word-granular accesses. Figure 3.2 shows the arrangement.

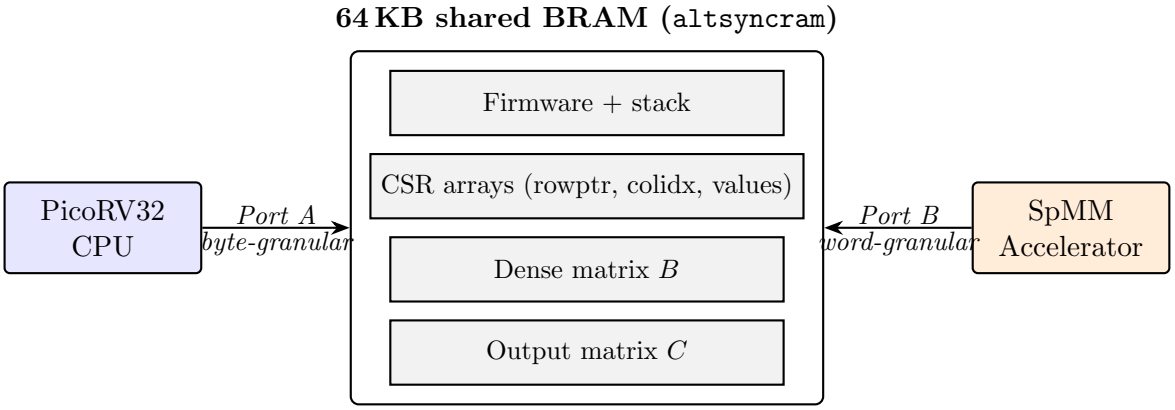

FIGURE 3.2: Dual-port BRAM organisation. The CPU and accelerator drive independent ports on the same memory array, so they can read or write different addresses on the same cycle without arbitration.

Because both ports address the same underlying array, same-word collisions are avoided in firmware: the CPU does not modify accelerator input buffers during execution and does not read C until completion is signalled.

3.4.3 Address Space

Table 3.2 summarises the full memory map.

Address range	Peripheral	Purpose
`0x0000_0000--0x0000_FFFF`	On-chip RAM (64 KB)	Firmware code, stack, CSR arrays (`rowptr`, `colidx`, `values`), dense matrix B, output matrix C
`0x1000_0000--0x1000_00FF`	SpMM accelerator MMIO	Configuration registers, control, and status
`0x2000_0000--0x2000_000F`	GPIO	LED output, switch and button input, seven-segment display
`0x2000_0100--0x2000_01FF`	UART (`simpleuart`)	Serial output for UART benchmark reporting at 115200 baud

TABLE 3.2: System memory map of the implemented RISC-V SoC.

3.5 Accelerator Programming Interface

The accelerator is configured through the MMIO registers in Table 3.3. Firmware writes the dimensions and base addresses, pulses `start`, polls `done`, and then reads

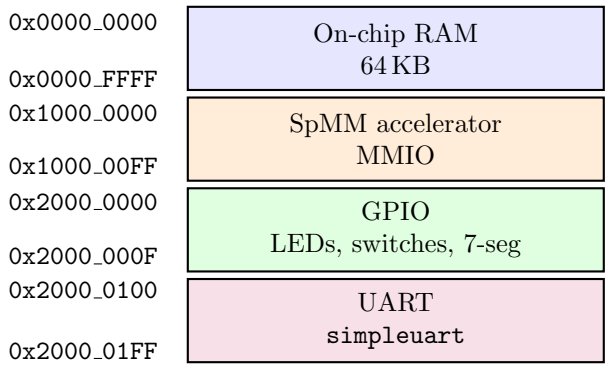

FIGURE 3.3: System memory map as a vertical address strip. The interconnect decodes each region using simple address comparisons.

the output from shared RAM. There is no DMA engine, interrupt controller, or driver layer.

Offset	Name	Description
0x00	CTRL	Control register: bit 0 = start, bit 1 = clear_done, bit 2 = irq_en, bit 3 = relu_en, bit 4 = int8_mode
0x04	STATUS	Status register: bit 0 = busy, bit 1 = done
0x08	M	Number of rows in A and C
0x0C	N	Number of columns in B and C (dense width)
0x10	K	Number of columns in A / rows in B
0x14	A_VAL_BASE	Base address of CSR values array
0x18	A_ROW_BASE	Base address of CSR rowptr array
0x1C	A_COL_BASE	Base address of CSR colidx array
0x20	B_BASE	Base address of dense matrix B (row-major)
0x24	C_BASE	Base address of output matrix C (row-major)
0x28	NNZ	Total number of nonzeros in A

TABLE 3.3: SpMM accelerator MMIO register map. All registers are 32-bit word-aligned. Base addresses are byte addresses into the shared 64 KB RAM.

3.6 System Operation

Figure 3.4 shows the handshake between firmware and accelerator for a single SpMM run. The CPU stages all inputs in BRAM, writes the MMIO configuration, pulses start, and then polls the status register. The accelerator runs asynchronously on Port B of the BRAM until it has written the last element of C, at which point it raises done. The CPU clears the flag and proceeds to read C.

On a start pulse, the accelerator latches the dimensions and base addresses, traverses A in CSR form, reads B from shared RAM, computes C tile by tile, optionally

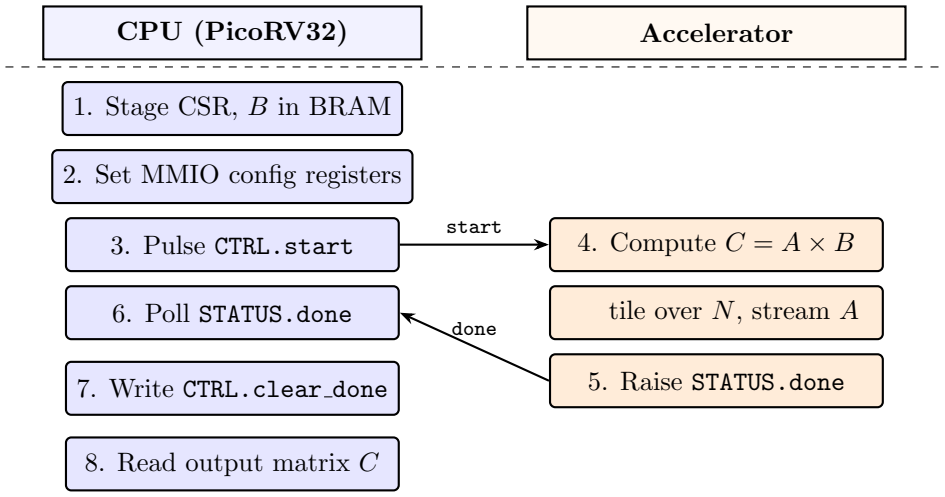

FIGURE 3.4: System operation sequence for a single SpMM run. Steps 1–4 and 7–8 execute on the CPU; steps 5–6 execute on the accelerator concurrently with the CPU's polling loop. The two lanes synchronise only through the `start` pulse and the `done` flag.

applies ReLU on writeback, and asserts `STATUS.done`. In INT8 mode, the `values` array and B matrix are interpreted as packed signed bytes and sign-extended before multiplication.

3.7 Design Constraints and Implications

Four constraints shaped the platform. First, all benchmark data lives in 64 KB of on-chip BRAM, which removes external-memory latency but limits practical problem size to roughly $M = K = 64$ and $N = 32$. Second, the accelerator must be driven from bare-metal firmware with no operating system, DMA engine, or interrupt controller, so a small polling-based MMIO interface was used. Third, the Cyclone V DSP, BRAM, and routing budget limits the practical tile width; in this design $TN = 8$ was the largest width that met timing reliably. Finally, all benchmarks must be reproducible, so the firmware uses fixed seeds, element-wise software checking, and PicoRV32's `rdcycle` counter.

Chapter 4

Accelerator Design

This chapter describes how the SpMM accelerator is built and why. It starts with a short overview of the architecture, works through the baseline datapath and the control FSM, and then covers the two design choices that do most of the work: the tile width and the row-wise CSR traversal. The final sections describe the DSP and INT8 optimisations and the physical-design constraints they exposed.

4.1 Overview

The accelerator has three jobs: walk the CSR structure of matrix A, stream row segments of matrix B into a local buffer, and accumulate products into a tile of the output matrix C. It splits into two co-operating modules:

- `accel_ctrl`: a finite state machine that walks the CSR indices, issues RAM requests, and sequences the datapath.
- `accel_datapath`: the MAC array and the two local buffers (`B_Seg` and `C_Tile`) that hold the working set of a single tile.

A top-level wrapper (`accel_top`) connects the two blocks through a small MMIO front-end that decodes the PicoRV32 bus and surfaces the configuration registers listed in Table 3.3. The accelerator reaches memory through Port B of the dual-port BRAM, which returns one 32-bit word per cycle with one cycle of registered read latency. Because that latency is fixed, every RAM access is scheduled deterministically and no stall/ready handshake is needed.

4.2 Baseline Datapath

The datapath is where the arithmetic happens. It is parameterised by three numbers: the tile width $TN = 8$ (the number of output columns processed in parallel), a maximum row count $M_{\max} = 64$, and a 32-bit signed operand width. Figure 4.1 shows the block diagram.

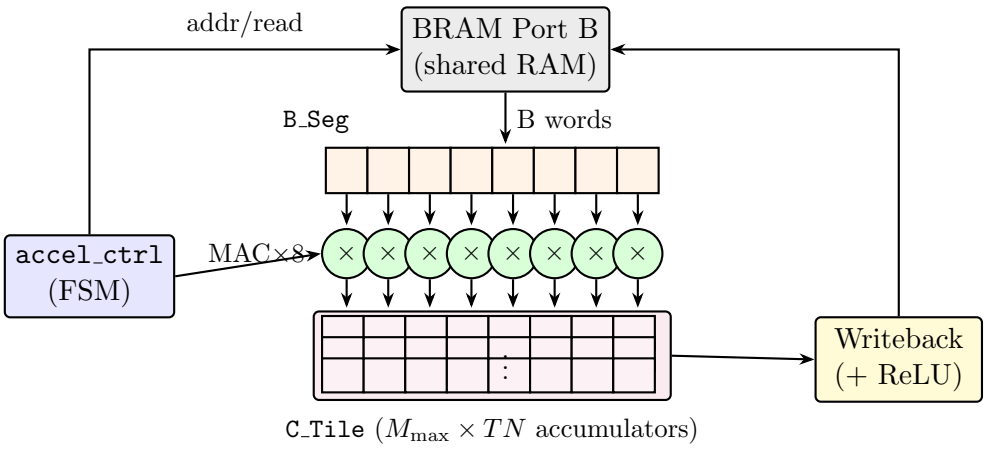

FIGURE 4.1: Baseline datapath. The controller fills `B_Seg` from Port B of the shared BRAM, drives the common operand `mac_a` to all eight MAC lanes, and selects the target row of `C_Tile`. Once a tile column is complete, the writeback path streams `C_Tile` back to memory, applying ReLU if enabled.

Two local buffers carry the state of a tile.

`B_Seg` is an eight-element array of 32-bit signed values that holds the row segment of B currently being consumed. For each nonzero a_{ij} in row i of A, the controller fills `B_Seg` with $B[j, c_0 : c_0{+}TN]$ before the MAC fires.

`C_Tile` is an $M_{\max} \times TN$ array of 32-bit signed accumulators that holds the partial sums for every row of C within the current tile column $[c_0, c_0{+}TN]$. It is implemented as fabric registers (around 256 ALMs on Cyclone V), not BRAM, so every entry can be read and updated on the same cycle. Keeping it flat, rather than as a set of per-row registers, matters because nonzeros from different rows often land in the same tile column and the flat layout lets those updates proceed back-to-back with no writeback/restore cycles between them.

The core operation is one line. For each nonzero $a = A[i, k]$ the controller asserts `mac_en`, drives `mac_row` $= i$ and `mac_a` $= a$, and the datapath issues eight parallel MACs in a single clock:

$$\texttt{C\_Tile}[i][c] \;\mathrel{+}=\; a \times \texttt{B\_Seg}[c], \qquad c = 0, 1, \ldots, TN{-}1.$$

Figure 4.2 shows the tile column inside C that this corresponds to.

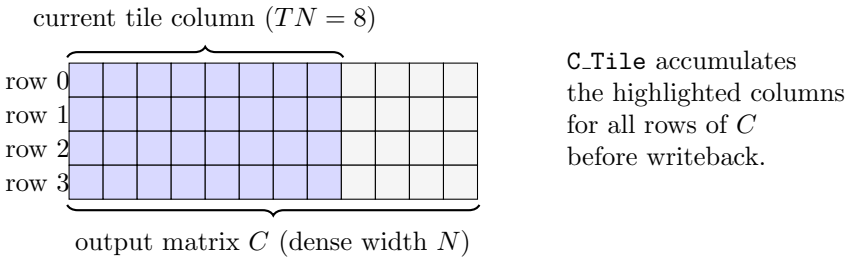

FIGURE 4.2: Tile column view. While the accelerator works on one tile column, the highlighted eight columns of C are held in `C_Tile` across all rows. Nonzeros from any row of A update their respective `C_Tile` row directly.

Once every row has been visited for the current tile column, the datapath walks `C_Tile` row by row and writes each entry back to shared RAM. If `CTRL.relu_en` is set, an element-wise $\max(0, x)$ is applied on the writeback path.

4.3 Control FSM

The control FSM drives the datapath. It generates every RAM address, tracks the CSR traversal, and decides when to fill `B_Seg`, fire a MAC, or flush `C_Tile` back to memory. Table 4.1 lists its states, and Figure 4.3 shows the transitions.

The FSM implements a tile-column outer loop, a row loop, and a nonzero loop, with explicit sub-phases to absorb the one-cycle BRAM latency.

A first-order cycle model per tile column is

$$T_{\text{tile}} \approx M_{\max} + M \cdot C_{\text{row}} + \text{NNZ} \cdot (C_{\text{nz}} + TN) + M \cdot TN \cdot C_{\text{wb}},$$

with $C_{\text{row}} \approx 5$, $C_{\text{nz}} \approx 5$, and $C_{\text{wb}} \approx 1$. The model is deliberately approximate: it ignores writeback overlap and assumes no stalls. Its purpose is to expose the weight of the $\text{NNZ} \cdot TN$ term. At $TN = 8$, filling `B_Seg` costs eight cycles per nonzero, which dominates the other terms in every realistic case. Section 4.6 explains why, and Section 4.8 attacks that term directly.

State	Action
`IDLE`	Wait for `start` pulse. Latch M, N, K, NNZ, and base addresses from the MMIO register snapshot.
`INIT_TILE_CLEAR`	Clear all entries of `C_Tile` to zero in preparation for the first tile column ($M_{\max}$ cycles, one row per cycle).
`ROW_LOAD`	Read the CSR row bounds `rowptr[i]` and `rowptr[i+1]` for the current row i. Sub-phases `ROW_PHASE_0/1/2` absorb the registered RAM latency.
`NZ_FETCH`	For each nonzero in `[rowptr[i], rowptr[i+1])`: read `values[p]` and `colidx[p]` and compute the starting address of B[`col`, :] for the current tile. Sub-phases `NZ_PHASE_0`–`NZ_PHASE_4` handle RAM latency and address arithmetic.
`MAC`	Fill `B_Seg` with TN consecutive values from B (one per cycle in INT32 mode, two reads total in INT8 mode), then assert `mac_en` for one cycle to update `C_Tile`.
`NEXT_ROW`	Increment the row index. If rows remain in this tile column, loop back to `ROW_LOAD`; otherwise proceed to `WRITE_TILE`.
`WRITE_TILE`	Write all $M \times TN$ entries of `C_Tile` back to shared RAM (with optional ReLU). Advance the tile-column offset by TN; if more tile columns remain, clear `C_Tile` and loop to `ROW_LOAD`, otherwise proceed to `DONE`.
`DONE`	Assert `STATUS.done` and release `STATUS.busy`; return to `IDLE` when `CTRL.clear_done` is asserted.

Table 4.1: Accelerator FSM states and their actions.

4.4 The Choice of Tile Width

This section explains why the tile width is set to eight. TN controls four things at once: how many MAC lanes run in parallel, how large the accumulator array is, how many RAM reads are needed to fill `B_Seg`, and how hard the design is to route at 50 MHz. Table 4.2 lists the alternatives.

$TN = 8$ was selected as the largest width that met 50 MHz timing in trial builds while keeping the `C_Tile` array and DSP usage manageable. It also covers the $N = 8$ benchmark cases in a single tile column.

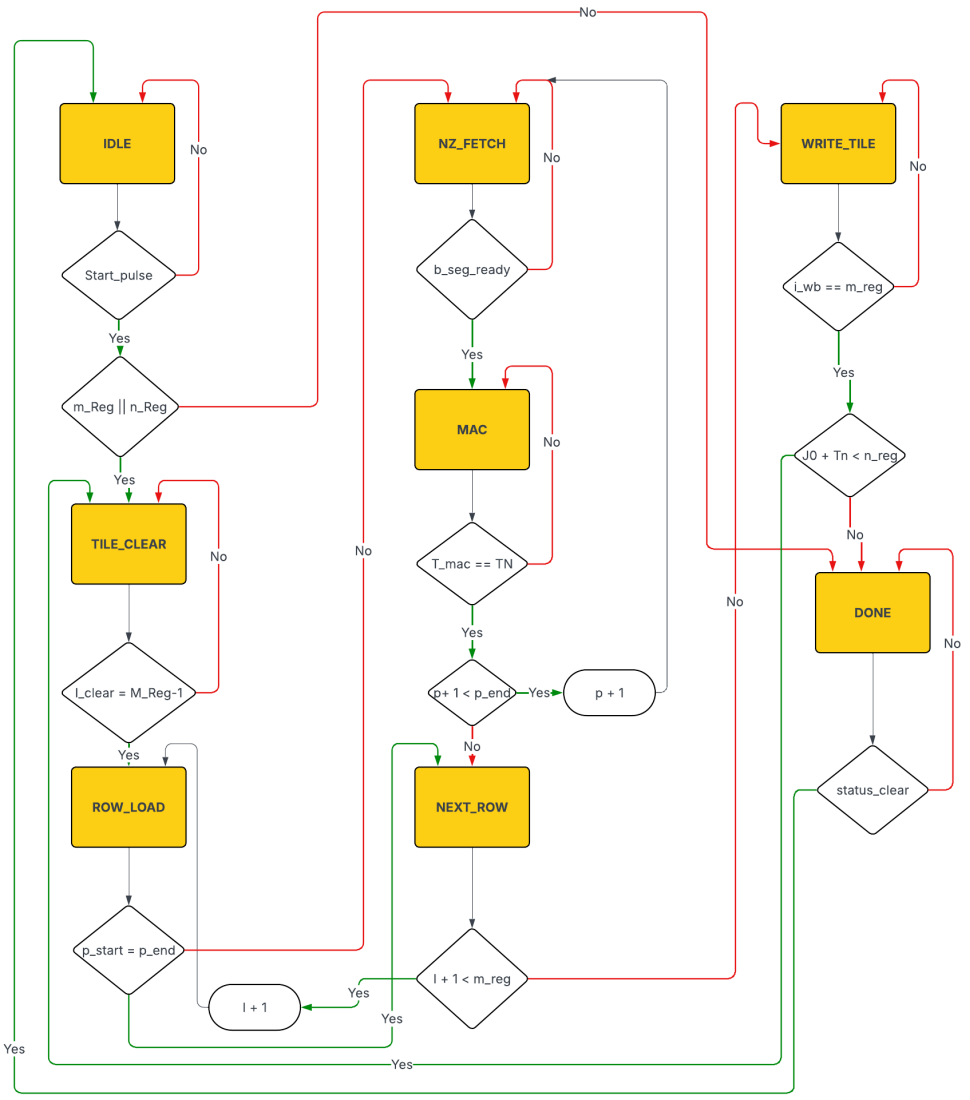

FIGURE 4.3: Simplified accelerator finite state machine. Each major phase corresponds to one row in Table 4.1. Sub-phase transitions within `ROW_LOAD` and `NZ_FETCH` absorb the one-cycle registered RAM output latency.

4.5 Algorithmic Decomposition: Row-Wise CSR Traversal

The accelerator uses row-wise CSR traversal because it minimises the control and storage structures needed on a small FPGA. More complex outer-product or intersection-based approaches, such as those used by OuterSPACE [8] or ExTensor [10], can improve throughput for some sparse workloads, but they require merge

TN	MAC lanes	Acc. registers	B-seg reads/NZ	DSP No.	Notes
1	1	64	1	1	Minimal resources; no parallelism; effectively sequential.
4	4	256	4	4	Moderate parallelism; reduced register pressure.
8	8	512	8	8	**Design point.** Good balance; DSP-mappable; 512-reg array feasible.
16	16	1024	16	16	Doubles resources; routing congestion; timing harder to close.
32	32	2048	32	32	Likely infeasible on Cyclone V; 2048-reg array exceeds practical limits.

TABLE 4.2: Resource and performance implications of different tile widths TN. The accelerator implements $TN = 8$.

networks, multi-port accumulation, or intersection hardware. These structures were not appropriate for the resource budget and verification scope of this SoC.

Row-wise CSR allows the controller to track one row pointer at a time, stream most CSR accesses sequentially through BRAM Port B, and write back `C_Tile` with a simple row sweep. This is the traversal order used in the implemented FSM.

4.6 Memory Optimisation: the B-Segment Bottleneck

This section explains why the baseline accelerator is memory-bound and why the B-segment fill is the dominant cost.

Operating at 50 MHz with $TN = 8$ parallel MAC lanes, the peak arithmetic throughput is

$$T_{\text{peak}} = 8 \text{ MACs/cycle} \times 50 \text{ MHz} = 400 \text{ MMACs/s}.$$

Port B of the BRAM supplies one 32-bit word per cycle, or about 200 MB/s of operand bandwidth. Filling a $TN = 8$ segment from 32-bit operands costs eight reads, so the MAC lanes wait for operands roughly eight times longer than they compute. The design sits well to the left of the roofline knee introduced in Section 2.8, and any optimisation that does not shrink the B-fill term is capped at a small fraction of the ideal speedup.

In the baseline, each nonzero takes about ten cycles of useful work: eight to fill `B_Seg`, plus one cycle of registered read latency on the first word, plus one MAC cycle. Of those ten, the eight fill cycles are entirely memory-bound.

4.7 DSP-Oriented Parallel MAC Enhancement

This section covers the first optimisation stage, which improves how the eight MAC lanes map onto Cyclone V DSP blocks.

Each DSP block on Cyclone V provides an 18×18-bit signed multiplier with a dedicated accumulator register [20]. Quartus will map an RTL multiplier onto a DSP block, but only when the operand widths and the accumulator structure match what the block expects. In the baseline the eight MACs are written as generic 32-bit multiplications, which Quartus may not map onto DSP blocks as predictably.

The DSP MAC stage rewrites each lane in the registered-feedback form that Quartus recognises:

$$\texttt{C\_Tile}[i][c] \leftarrow \texttt{C\_Tile}[i][c] + a \cdot B[c],$$

with 32-bit signed operands and a registered accumulator. With this form, the MAC lanes are designed to map more predictably onto dedicated DSP multiply-accumulate resources, reducing the amount of arithmetic implemented in ALM logic. However, this stage does not change the number of B-segment reads per nonzero, so it cannot remove the dominant memory-side bottleneck identified in Section 4.6. The larger performance opportunity therefore comes from reducing the B-fill cost, which is the purpose of the INT8 packed mode in Section 4.8.

4.8 INT8 Mode at the Datapath Level

This section describes what changes in the datapath when INT8 mode is enabled. When `CTRL.int8_mode` is set, three specific things change:

1. **The CSR `values` array is packed as four INT8 per 32-bit word.** For nonzero index p the byte lives at `A_VAL_BASE` $+\lfloor p/4 \rfloor \times 4$, and the controller tracks the byte offset $b = p \bmod 4$ with a two-bit counter.
2. **Matrix B is packed the same way.** The eight values needed to fill `B_Seg` now come from two aligned 32-bit reads instead of eight.
3. **Each selected byte is sign-extended to 32 bits** in a single combinatorial stage before it reaches the multiplier:

$$\texttt{B\_Seg}[4k+b] = \text{sign\_ext}_{8\to32}(\texttt{ram\_rdata}[8(b{+}1){-}1 : 8b]), \quad b \in \{0,1,2,3\}. \tag{4.1}$$

Figure 4.4 shows the byte-select and sign-extend path for one 32-bit read.

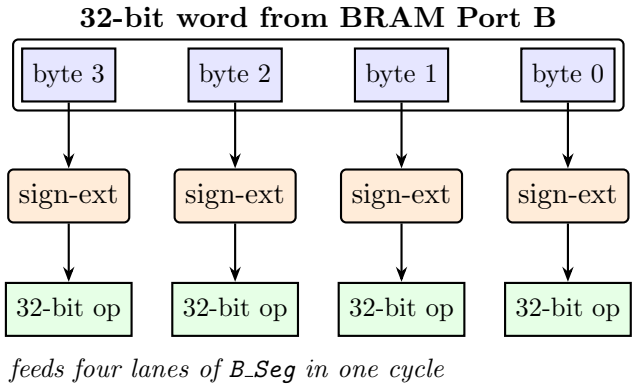

FIGURE 4.4: INT8 byte-select and sign-extension path. One 32-bit read delivers four INT8 operands; each is sign-extended to 32 bits in a single combinatorial stage before it reaches a MAC lane. Two such reads fill all eight lanes of `B_Seg`.

Accumulation and writeback remain INT32, so the change is confined to the memory-facing unpack/sign-extension path.

4.9 Physical Design Tradeoffs

This section covers the practical FPGA constraints that shaped the design beyond the RTL-level decisions above.

Adding the C-tile register array, the B-segment buffer, and eight DSP-mapped MACs raises the fan-out of several controller signals. The `mac_a` operand in particular has to drive all eight multiplier inputs at once. On Cyclone V, a high-fan-out register combined with the wide BRAM port, the CSR-address paths, and the tile writeback network creates routing congestion that does not appear in RTL simulation.

The target clock is 50 MHz, a comfortable 20 ns period. Cyclone V designs with DSP-mapped multipliers routinely reach 100–200 MHz, so the multiplier is not the critical path. The critical path is the combinatorial logic that generates CSR addresses and drives eight parallel writes into `C_Tile`. After the INT8 packing datapath was introduced, one synthesis run failed to close timing under the default balanced optimisation mode and required the Quartus fitter to be switched to performance-oriented optimisation.

Table 4.3 summarises what changed across the three stages. The MAC width, accumulator precision, and output format stay the same (8 lanes, INT32, INT32). The DSP mapping and the per-nonzero B-fill cost are the only things that vary, and those two knobs account for the cycle reduction from baseline to final INT8 build.

Attribute	Baseline	DSP MAC	INT8 packed
MAC lanes	8	8	8
DSP mapping	Inferred	Explicit	Explicit
B reads per NZ	8	8	2
Operand bit-width	32-bit	32-bit	8-bit (sign-ext to 32)
Accumulator precision	INT32	INT32	INT32
Output format	INT32	INT32	INT32

TABLE 4.3: Architectural attributes across the three design stages. Measured speedups for each stage are reported in Chapter 6.

Chapter 5

Hardware Verification

5.1 Verification Approach

Verification was carried out at two complementary levels: RTL-level simulation before synthesis, and hardware-level functional checking on the DE1-SoC after programming. This two-level approach is standard in digital design practice [21]. RTL simulation is fast and exposes every internal signal, but it cannot by itself guarantee that synthesis, placement, and routing have preserved functional behaviour; hardware-level checking closes that gap by exercising the complete SoC on the physical device. A discrepancy at either level is sufficient to reject a result, and every cycle count reported in Chapter 6 comes from a run that passed both. Figure 5.1 summarises the overall flow.

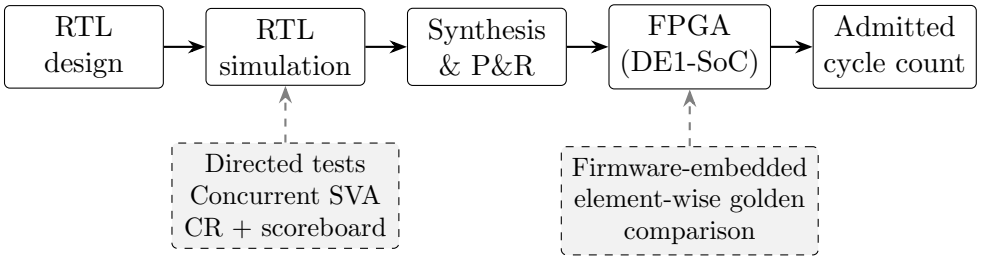

FIGURE 5.1: Two-level verification flow. Each transition between stages is gated by its associated check set; only cycle counts from runs that pass every applicable check are admitted as results.

5.2 RTL Simulation

The simulation tier combines the three verification styles typical of industry-grade digital flows: directed tests with hand-computed golden outputs, concurrent SystemVerilog Assertions (SVA) for always-on protocol and liveness monitoring, and constrained-random stimulus paired with a software scoreboard for exploring the input space beyond what directed tests reach. Four SystemVerilog testbenches share a common stimulus and check framework, three at the unit level and one at the full-SoC integration level. Their scope and the methods applied in each are summarised in Table 5.1.

Testbench	Scope	Methods applied
Datapath unit TB	Arithmetic core in isolation	Directed, CR + scoreboard, SVA
Control-FSM unit TB	FSM with RAM model and datapath stub	Directed, SVA
Accelerator-top TB	MMIO, FSM and datapath together	Directed, CR + scoreboard, SVA, coverage
Integration TB	PicoRV32, accelerator and 64 KB RAM	Firmware-driven, SVA

TABLE 5.1: The four SystemVerilog testbenches that make up the RTL simulation tier, their DUT scope, and the verification methods applied in each.

5.2.1 Unit testbenches

The datapath testbench drives the arithmetic core in isolation, covering B-segment load, MAC accumulation, C-tile clear, signed-operand behaviour, and ReLU on positive, zero, and negative inputs; a constrained-random test then randomises twenty MAC operations per iteration and compares the full tile against a software scoreboard after each batch. The control-FSM testbench instantiates the controller together with a one-cycle-latency RAM model and a datapath stub, and verifies that the FSM enters and exits its busy region within a bounded number of cycles and that its RAM-side signals honour single-port, alignment, and reset-quiescence invariants. The accelerator-top testbench closes the loop between the MMIO front-end, the FSM, and the datapath; a transaction object with random M, K, N, and ReLU enable drives twenty end-to-end runs per seed, each read back and compared to its expected output, with a functional covergroup recording which configuration combinations have been exercised.

5.2.2 Assertion-based checking

Every testbench carries a fixed library of concurrent SVA properties that guard invariants holding for the entire simulation rather than for any one stimulus. These invariants fall into five categories: mutual exclusion of the datapath command signals (the clear, *B*-segment write, MAC, and writeback strobes cannot fire simultaneously); 32-bit address alignment on every RAM access; quiescence of the RAM interface during reset; exclusivity between the `BUSY` and `DONE` status bits; and liveness, which requires every rising edge of `BUSY` to be followed by a falling edge within a bounded number of cycles. The same SVA set is shared across the FSM, accelerator-top, and integration testbenches, so any RTL change that breaks one of these contracts is caught by the entire suite at once.

5.2.3 Constrained-random stimulus and scoreboarding

The constrained-random tests randomise transaction objects whose fields cover the legal configuration space, drive the DUT, and compare the resulting output against a parallel software model updated in lock-step. A functional covergroup samples the exercised configurations after each iteration, so coverage convergence is visible during regression and seed counts can be extended until every bin is hit. The scoreboard itself is an element-wise check over the DUT's full output; any mismatch is reported with the exact index of the offending element and halts the test.

5.2.4 Integration testbench

The integration testbench instantiates PicoRV32, the accelerator, and a 64 KB behavioural RAM, preloads the compiled firmware image, releases reset, and lets the firmware execute its complete benchmark sequence until the CPU writes a sentinel value to a pre-agreed address. This closes the loop between the software control flow of Chapter 6 and the hardware described in Chapter 4: any firmware change that breaks the MMIO contract, any RTL change that violates the firmware's timing assumptions, or any memory-map misconfiguration fails immediately in simulation, before reaching a synthesis run. Additional SVA properties guard the shared memory fabric, most importantly address-decode exclusivity between the RAM and MMIO regions and bounded progress on the CPU memory handshake.

5.3 Hardware-Level Verification

Hardware-level verification is embedded directly inside the firmware benchmark harness described in Section 6.1.1. For each of the 12 cases and at each of the three accelerator stages, firmware generates deterministic inputs from fixed seeds, computes the quantised CPU reference, runs the accelerator on the same inputs, and performs an element-by-element comparison between the accelerator output and the software reference. Because CPU and accelerator apply the same symmetric INT8 quantisation to the same data and accumulate identical INT32 partial sums, their outputs must agree exactly; a single mismatch flags the case as FAIL and no cycle count is recorded.

Taken together with the simulation tier, this yields three independent layers of functional checking before any performance measurement is admitted. Concurrent SVA guards low-level RTL contracts; randomised scoreboards probe the configuration and operand space; and element-wise golden comparison on real silicon certifies that the synthesised, placed, and routed design produces the correct output under real timing. The benchmark inputs, cycle-count definitions, and the resulting measurements are treated as evaluation methodology and are presented in Chapter 6.

Chapter 6

Firmware, Benchmarking and Evaluation

This chapter is the project's experimental half. It describes the bare-metal firmware that drives the accelerator, the benchmark suite that exercises it, and the measured results produced on hardware. The order follows the order of the work: the firmware was built first, the benchmark cases were designed once the firmware was stable, and the cycle-count and correctness numbers reported at the end were produced by running that firmware and that suite on the DE1-SoC.

6.1 Firmware Benchmark Harness

The bare-metal firmware drives the entire benchmark loop: matrix generation, data preparation, accelerator configuration, timing, correctness checking, and result reporting. It runs directly on PicoRV32 from address `0x0000_0000` with no operating system. A short RISC-V assembly bootstrap sets the stack pointer to the top of the 64 KB RAM, clears the BSS section, and enters `main()`, which iterates over the 12 test cases and, for each, drives matrix generation, the software reference, the accelerator launch, correctness comparison, and UART reporting. The driver wraps the MMIO register interface behind configuration, start, and polling functions; libc stubs (`memset`, `memcpy`, `memmove`) cover the minimal C library needs. The linker script maps all code and data into the same 64 KB RAM, with the stack growing downward from the top.

Matrix buffers are allocated statically below the stack, so the CSR arrays, dense matrix B, and the output matrices (`c_accel`, `c_cpu`) occupy contiguous regions. The largest benchmark (T6: $64 \times 64 \times 32$, 75% sparsity, NNZ ≈ 1024) consumes roughly

$$\underbrace{65 \times 4}_{\texttt{rowptr}} + \underbrace{1024 \times 4}_{\texttt{colidx}} + \underbrace{1024 \times 4}_{\texttt{values}} + \underbrace{64 \times 32 \times 4}_{B} + \underbrace{64 \times 32 \times 4}_{C} \approx 29\,\text{KB}$$

of matrix data, well within the 64 KB budget for firmware code and stack.

6.1.1 Runtime data preparation

Dense matrix B and sparse matrix A are generated at runtime from fixed per-test seeds, so the same inputs are used across all optimisation stages. Values are sampled in $[-127, 127]$, which can be used directly by the INT32 path and then quantised for the INT8 path.

The sparse generator supports the three patterns used in the benchmark suite: uniform, row-skewed, and clustered. It chooses nonzero positions, assigns signed values, sorts column indices within each row for determinism, and writes the resulting canonical CSR arrays (`rowptr`, `colidx`, and `values`). The pattern details matter for interpreting T8, T10, and T11, but the exact PRNG sequence is not part of the assessed result.

6.1.2 INT8 quantisation and packing

Before an INT8 run, firmware applies symmetric runtime quantisation separately to the CSR values array and to B. For a vector $\mathbf{v}$ of length L, it computes $v_{\max} = \max_i |v_i|$, sets $s = v_{\max}/127$, and quantises each element as $q_i = \text{clip}(\text{round}(v_i/s), -127, 127)$. When $v_{\max} = 0$, all outputs are zero and $s = 1$. This matches Equation 2.1 from Section 2.7. The scale factor is discarded immediately: because the correctness check compares accelerator output against the *quantised* CPU reference, s need never be stored, which keeps both the accumulator logic and the verification path scale-free.

After quantisation, four consecutive INT8 values are packed into a single 32-bit word in little-endian byte order:

```
for (i = 0; i < len; i += 4) {
    word = ((uint32_t)(uint8_t)q[i+0])        |
           ((uint32_t)(uint8_t)q[i+1] << 8)   |
           ((uint32_t)(uint8_t)q[i+2] << 16)  |
           ((uint32_t)(uint8_t)q[i+3] << 24);
    dst[i/4] = word;
}
```

B is packed row by row, so a single 32-bit RAM read delivers four aligned INT8 values that populate four lanes of the B-segment buffer after sign extension, as described in Section 4.3.

6.1.3 Software baseline

The software reference is deliberately written in the simplest possible form. The full-precision inner structure is:

```
for (row = 0; row < M; row++) {
    for (p = rowptr[row]; p < rowptr[row+1]; p++) {
        col = colidx[p];
        a   = values[p];
        for (n = 0; n < N; n++)
            C[row*N + n] += a * B[col*N + n];
    }
}
```

Listing 6.1.3: PicoRV32 software SpMM (full precision).

Only the M-extension `MUL` instruction is used; there is no SIMD, no loop unrolling, and no software prefetching. The baseline is deliberately simple: its role is not to maximise CPU throughput but to provide a credible, readable reference against which the accelerator's structural advantage can be measured. The INT8 reference (`spmm_cpu_q8`) has the same structure but operates on signed 8-bit operands with INT32 accumulation, and is the version used for correctness comparison against the INT8 accelerator.

6.1.4 Accelerator launch and reporting

The driver exposes three entry points: `accel_configure()` writes the dimension and base-address registers, `accel_start_mode()` issues the control write, and `accel_wait_done()` polls `STATUS.done`. Each benchmark clears the done flag, configures the accelerator, records t_0, starts the run, polls for completion, records t_1, and reports $t_1 - t_0$.

Results are emitted as CSV over the on-chip UART at 115200 baud (50 MHz / 434 cycles per bit); each line carries the test ID, M, K, N, sparsity percentage and pattern, NNZ count, CPU and accelerator cycle counts, derived speedup, and a PASS/FAIL verdict.

Correctness is checked inline after each accelerator run by comparing `c_accel` against the INT8 CPU reference `c_cpu` element by element:

```
for (i = 0; i < M*N; i++) {
    if (c_accel[i] != c_cpu[i]) {
        pass = 0; break;
    }
}
```

A full-precision CPU result is also computed in parallel but untimed, and serves as a sanity reference for inspecting quantisation error. Because CPU and accelerator apply the same symmetric INT8 quantisation to the same data and accumulate identical INT32 partial sums, their outputs must match exactly; any mismatch indicates a hardware bug rather than a numerical artefact.

6.2 Benchmark Suite

The suite sweeps four independent axes so that any observed trend can be interpreted analytically rather than anecdotally: matrix size, which governs amortisation of fixed setup overhead; dense width N, which sets the number of tile columns and the ratio of tile-management to useful work; sparsity, which determines NNZ and therefore per-row arithmetic load; and nonzero distribution pattern, which exposes sensitivity to the spatial layout of NNZ. Table 6.1 lists all 12 cases with the axis each primarily addresses. The same cases are used unchanged across all three stages; this is what makes stage-to-stage comparison meaningful.

Test ID	$M \times K \times N$	Sparsity	Pattern	Purpose
T1	$8 \times 8 \times 8$	75%	Uniform	Small debug/sanity case; establishes minimum speedup in the suite.
T2	$16 \times 16 \times 8$	75%	Uniform	Small size-scaling point; 4× more arithmetic than T1.
T3	$32 \times 32 \times 8$	75%	Uniform	Medium size-scaling point.
T4	$64 \times 64 \times 8$	75%	Uniform	Largest square case; maximum M for the current $M_{\max} = 64$.
T5	$64 \times 64 \times 16$	75%	Uniform	Tests effect of doubling dense width relative to T4.
T6	$64 \times 64 \times 32$	75%	Uniform	Tests effect of quadrupling dense width; 4 tile columns.
T7	$32 \times 32 \times 32$	50%	Uniform	Low sparsity (dense); more nonzeros per row than T8/T9.
T8	$32 \times 32 \times 32$	75%	Uniform	Mid-sparsity reference; matches T10/T11 except pattern.
T9	$32 \times 32 \times 32$	90%	Uniform	Very sparse; exposes overhead-to-arithmetic ratio.
T10	$32 \times 32 \times 32$	75%	Row-skewed NNZ	Same NNZ as T8; row imbalance sensitivity.
T11	$32 \times 32 \times 32$	75%	Clustered NNZ	Same NNZ as T8; block locality sensitivity.
T12	$64 \times 64 \times 32$	90%	Uniform	Large sparse stress case; wide output and few nonzeros.

TABLE 6.1: Complete benchmark input set used across all recorded evaluation stages.

6.3 Cycle-Count Definitions and Measurement Method

CPU cycle counts bracket only the software SpMM kernel: matrix generation, quantisation, and output clearing are excluded. Accelerator cycle counts bracket the complete offload call, including MMIO configuration, the `start` pulse, hardware execution, and polling. The accelerator number is therefore end-to-end offload latency rather than a pure datapath-throughput figure.

Timing uses the PicoRV32 `rdcycle` CSR, which returns the lower 32 bits of a hardware cycle counter clocked at 50 MHz. The read is wrapped in a short inline-assembly stub:

```
static inline uint32_t read_cycles(void) {
    uint32_t cyc;
    asm volatile("rdcycle %0" : "=r"(cyc));
    return cyc;
}
```

The counter wraps at 2^{32} cycles, roughly 85 seconds at 50 MHz. The largest benchmark completes in under two million cycles, so wrap-around is a non-issue and firmware needs no 32-to-64-bit extension.

Speedup is defined as

$$\text{Speedup} = \frac{T_{\text{CPU}}}{T_{\text{ACCEL}}},$$

with both times in CPU cycles on the same 50 MHz clock. For the RISC-V comparisons, T_{CPU} is the timed PicoRV32 software result (INT32 for the baseline and DSP stages, INT8 for the INT8 stage). For the Cortex-M0 comparison, T_{CPU} is derived from the processor's published CPI model applied to the same benchmark definitions rather than measured on physical hardware; Section 6.8 discusses the implications.

6.4 Baseline Accelerator Results

Table 6.2 gives the full baseline accelerator results against the PicoRV32 software reference. Even before the DSP and INT8 stages, the accelerator is consistently faster, with speedup rising from 12.12× at T1 to 18.12× at T4. The smallest case is limited by fixed FSM overhead, which is amortised better as matrix size increases.

Test ID	$M \times K \times N$ / Sparsity	CPU cycles	Accel cycles	Speedup	Result
T1	$8 \times 8 \times 8$, 75%	10,373	856	12.12×	PASS
T2	$16 \times 16 \times 8$, 75%	36,573	2,318	15.78×	PASS
T3	$32 \times 32 \times 8$, 75%	136,877	7,809	17.53×	PASS
T4	$64 \times 64 \times 8$, 75%	529,101	29,195	18.12×	PASS
T5	$64 \times 64 \times 16$, 75%	986,829	58,112	16.98×	PASS
T6	$64 \times 64 \times 32$, 75%	1,902,285	115,980	16.40×	PASS
T7	$32 \times 32 \times 32$, 50%	951,213	58,112	16.37×	PASS
T8	$32 \times 32 \times 32$, 75%	491,693	30,470	16.14×	PASS
T9	$32 \times 32 \times 32$, 90%	215,263	13,844	15.55×	PASS
T10	$32 \times 32 \times 32$, 75%, row-skewed	491,693	30,470	16.14×	PASS
T11	$32 \times 32 \times 32$, 75%, clustered	491,693	30,470	16.14×	PASS
T12	$64 \times 64 \times 32$, 90%	800,155	49,663	16.11×	PASS

TABLE 6.2: Baseline PicoRV32 software versus baseline accelerator results. Speedup ranges from 12.12× to 18.12× across the 12-case suite.

6.5 Parallel DSP MAC Enhancement Results

Table 6.3 gives the results after the DSP-oriented MAC restructuring described in Chapter 4. Total accelerator cycles fall from 427,299 to 324,568, a 1.32× improvement, with every case improving over its baseline counterpart. The gain is useful but modest, showing that arithmetic scheduling alone does not remove the dominant memory-side cost.

Test ID	CPU cycles	Accel cycles	Speedup	Result
T1	10,373	737	14.07×	PASS
T2	36,573	1,859	19.67×	PASS
T3	136,877	6,024	22.72×	PASS
T4	529,101	22,021	24.03×	PASS
T5	986,829	43,781	22.54×	PASS
T6	1,902,285	87,301	21.79×	PASS
T7	951,213	43,781	21.73×	PASS
T8	491,693	23,296	21.11×	PASS
T9	215,263	10,988	19.59×	PASS
T10	491,693	23,296	21.11×	PASS
T11	491,693	23,296	21.11×	PASS
T12	800,155	38,188	20.95×	PASS

TABLE 6.3: Results after enabling the parallel DSP-oriented MAC enhancement. Total accelerator cycles fell from 427,299 to 324,568, a 1.32× reduction.

6.6 INT8 Packing Results

Table 6.4 gives the final INT8 results, the largest performance step in the project. Total accelerator cycles fall from 324,568 to 148,737, a 2.18× reduction, and the best case reaches 56.80× over the quantised PicoRV32 reference. The gain comes from packing four INT8 operands per 32-bit word, while row loads, index fetches, and writeback remain fixed costs.

Test ID	CPU cycles (INT8)	Accel cycles	Speedup	Result
T1	10,735	567	18.93×	PASS
T2	38,087	1,111	34.28×	PASS
T3	142,999	2,964	48.25×	PASS
T4	553,655	9,747	**56.80**×	PASS
T5	1,044,151	19,216	54.34×	PASS
T6	2,025,143	38,171	53.05×	PASS
T7	1,012,631	19,216	52.70×	PASS
T8	522,391	11,039	47.32×	PASS
T9	227,481	6,109	37.24×	PASS
T10	522,391	11,039	47.32×	PASS
T11	522,391	11,039	47.32×	PASS
T12	849,333	18,519	45.86×	PASS

TABLE 6.4: Final INT8-packed implementation results. Peak speedup of 56.80× on T4; average speedup of 45.28× across the 12-case suite. Total accelerator cycles fell from 324,568 to 148,737, a 2.18× reduction over the DSP stage.

6.7 Stage-Level Summary

Table 6.5 consolidates the per-stage results into a single view. All 12 benchmark cases pass their element-by-element correctness checks at every stage, on hardware. Each optimisation improves performance without a correctness regression. The stage totals also show that reducing operand traffic through INT8 packing was more valuable than the DSP arithmetic restructuring.

Stage	Avg speedup	Best case	Total accel cycles	Reduction vs prev.
Baseline accelerator	16.12×	T4: 18.12×	427,299	n/a
Parallel DSP MAC	20.87×	T4: 24.03×	324,568	1.32×
INT8 packed	45.28×	T4: 56.80×	148,737	2.18×

TABLE 6.5: Summary of the three staged RISC-V accelerator evaluations across the full 12-case benchmark suite.

6.8 Comparison Against ARM Cortex-M0

Table 6.6 compares the INT8 accelerator against a model-based ARM Cortex-M0 software reference, not a measurement on physical Cortex-M0 hardware. The model applies published single-issue CPI behaviour to the same benchmark definitions, giving a stronger embedded baseline than PicoRV32. Speedups are lower as expected, but the accelerator still reaches 11.92× on T4. These figures are a cross-check; the PicoRV32 hardware measurements remain the primary evidence.

Test ID	Cortex-M0 cycles	Accel cycles	Speedup	Result
T1	2,952	567	5.21×	PASS
T2	9,448	1,111	8.50×	PASS
T3	31,912	2,964	10.77×	PASS
T4	116,200	9,747	**11.92**×	PASS
T5	207,592	19,216	10.80×	PASS
T6	390,376	38,171	10.23×	PASS
T7	195,240	19,216	10.16×	PASS
T8	104,488	11,039	9.47×	PASS
T9	46,892	6,109	7.68×	PASS
T10	99,748	11,039	9.04×	PASS
T11	102,209	11,039	9.26×	PASS
T12	169,710	18,519	9.16×	PASS

TABLE 6.6: Model-based ARM Cortex-M0 software SpMM reference versus the final INT8 accelerator. The Cortex-M0 is a stronger baseline than PicoRV32, so speedups are correspondingly lower; the accelerator still achieves up to 11.92× speedup.

6.9 Result Analysis

6.9.1 Scaling with problem size (T1–T4)

Figure 6.1 plots speedup against matrix size for the three stages. Speedup rises consistently from T1 to T4 in all three, and in the INT8 stage grows from 18.93× at T1 ($8 \times 8 \times 8$) to 56.80× at T4 ($64 \times 64 \times 8$). The curve flattens as fixed setup costs become a smaller fraction of the total but do not disappear. T1 has too few nonzeros to hide tile clear and row-management overhead; T4 spreads those costs over far more useful MAC work.

6.9.2 Effect of dense width (T4–T6)

The sequence T4 ($N = 8$), T5 ($N = 16$), T6 ($N = 32$) tests how speedup changes as more tile columns are processed at fixed $M = K$. In the INT8 stage speedup changes only modestly, from 56.80× to 53.05×. Both CPU work and accelerator work scale roughly linearly with N, so the structural advantage is preserved. The slight decline comes from repeating tile clear and writeback for additional tile columns.

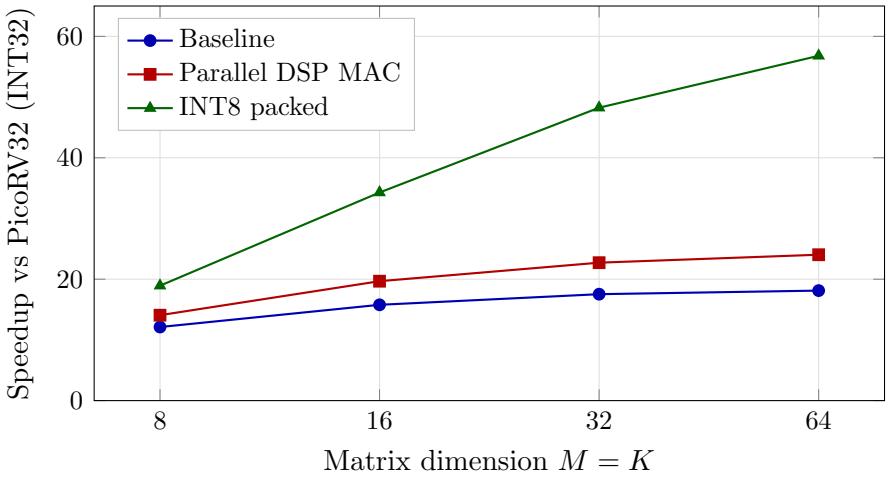

FIGURE 6.1: Speedup versus matrix size for tests T1–T4 (fixed $N = 8$, 75% uniform sparsity). Larger matrices amortise fixed setup overhead, with the INT8 stage giving the largest absolute gain.

6.9.3 Effect of sparsity (T7–T9)

The sequence T7 (50%), T8 (75%), T9 (90%) fixes $M = K = N = 32$ and sweeps sparsity, giving $52.70\times \rightarrow 47.32\times \rightarrow 37.24\times$ in the INT8 stage. Speedup falls as the matrix becomes sparser because per-row accelerator overhead remains even when rows contain few or no nonzeros, while the CPU loop skips those rows cheaply. Even at 90% sparsity the design still reports $37.24\times$ speedup, but the trend shows why extremely sparse real graphs would need a more aggressive empty-row bypass. Figure 6.2 plots the three-stage trend.

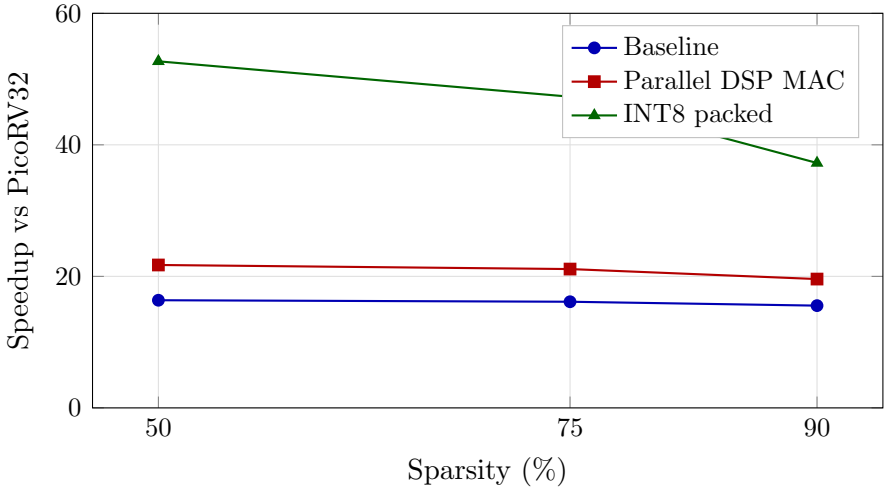

FIGURE 6.2: Speedup versus sparsity for tests T7–T9 (fixed $M = K = N = 32$, uniform pattern). The INT8 stage falls most sharply because fixed per-row traversal becomes a larger share of total runtime at high sparsity.

6.9.4 Pattern sensitivity (T8, T10, T11)

T8, T10, and T11 share dimensions, sparsity, and NNZ, differing only in how nonzeros are distributed (uniform, row-skewed, clustered). In the INT8 stage all three report identical accelerator cycle counts of 11,039 and CPU counts of 522,391. At this scale, total NNZ dominates cycle count more than spatial distribution. Larger matrices may expose load imbalance from very dense rows, but the present benchmark sizes do not trigger that effect. The Cortex-M0 model shows small differences between the same three patterns, so pattern sensitivity remains a useful future measurement target.

6.10 Resource and Timing Evaluation

Table 6.7 gives estimated resource utilisation for the three build variants. These are pre-place-and-route estimates from the Quartus Fitter combined with known design parameters, pending final post-fit reports. Utilisation is modest: roughly 16% of ALMs, 5% of M10K BRAMs, and 9% of DSP blocks. The 16 M10Ks implement the shared 64 KB RAM, while the 8 DSPs match the $TN = 8$ MAC lanes. All three builds meet the 50 MHz timing constraint.

Build	ALMs	Registers	M10K BRAMs	DSP blocks	Fmax (MHz)
Baseline accelerator	≈4,800	≈7,500	16	8	50+
Parallel DSP MAC	≈5,000	≈7,800	16	8	50+
INT8 packed (current)	≈5,200	≈8,000	16	8	50+
Available (Cyclone V)	32,070	128,280	397	87	n/a

TABLE 6.7: Estimated resource utilisation for the three build variants on the Cyclone V 5CSEMA5F31C6. All three designs meet the 50 MHz timing constraint. Final post-fit numbers to be updated in the next revision.

6.11 Fairness, Limitations, and Threats to Validity

The comparison is controlled in the main respects: CPU and accelerator use identical generated inputs, share the same 64 KB RAM and 50 MHz clock, and every timed accelerator result is checked against a software reference. The PicoRV32

baseline uses hardware `MUL`, but it is still a simple in-order core with no cache, speculation, SIMD, or aggressively tuned software kernel, so the headline speedups should not be generalised to modern vector CPUs. The Cortex-M0 comparison is model-based and should be read only as a cross-check, not as a same-platform measurement. Other limits are that each cycle count is a single deterministic run rather than a distribution, the matrices are synthetic rather than real workload traces, all data is held in on-chip BRAM rather than DRAM, MMIO/polling overhead is included in the accelerator timings, and final post-fit resource reports remain to be substituted for the estimates in Table 6.7.

6.12 Critical Evaluation

The staged results support the design argument: DSP-oriented arithmetic reduced total accelerator cycles by 1.32×, while INT8 packing reduced them by 2.18×. Direct comparison with large research accelerators such as OuterSPACE [8], SpArch [9], or SIGMA [11] would be misleading because they target different scales and memory systems. The relevant conclusion is narrower: on this DE1-SoC platform, a small MMIO-driven CSR SpMM accelerator achieves a 56.80× peak and 45.28× average speedup over PicoRV32, and up to 11.92× over the modelled Cortex-M0 baseline. With the limitations in Section 6.11, the highest-leverage optimisation was clearly reducing operand traffic through INT8 packing.

Chapter 7

Reflection

7.1 What Worked Well

Structuring development as a sequence of discrete, measurable optimisation stages (baseline, DSP MAC, and INT8 packing) was the single most valuable decision in the project. A working baseline was established and verified on hardware before any optimisation was attempted, and each subsequent stage changed exactly one aspect of the design: arithmetic parallelism first, then memory traffic. This made it possible to attribute each measured improvement to a specific architectural change, to detect regressions immediately, and to build the evidence-based narrative used throughout Chapter 6.

The deterministic benchmark harness also helped because fixed seeds and on-board golden checks made regressions immediately visible after each rebuild.

The roofline analysis in Chapter 2 was useful because it prevented the project from over-focusing on wider MAC arrays; the later INT8 result confirmed that memory traffic was the higher-leverage bottleneck.

7.2 What Went Wrong

The most time-consuming debugging issue was a stale `firmware.mif` file. The board kept running an old firmware image even after the C code had been rebuilt, which made correct RTL look faulty. The lesson was that software and hardware

artefacts must be tied together explicitly in the build system; a Makefile dependency on `firmware.mif` would have prevented this.

Timing closure also became harder as parallel logic was added. The INT8 datapath increased routing pressure around the C-tile and MAC buses, and Quartus only closed timing after switching to performance-oriented fitting. This showed that FPGA parallelism is limited not only by ALMs and DSPs, but also by placement and routing.

7.3 Project Management

The implementation and integration phases overran their planned allocation, mainly because the originally specified interrupt-driven completion path from the accelerator to the CPU proved too complex to integrate with the PicoRV32 exception model and was descoped in favour of a simple polling loop on the status register. In Semester 2, benchmarking was delayed when the UART peripheral bundled with the PicoRV32 reference design turned out not to support the cycle-accurate timing readouts required by the measurement methodology in Chapter 5; adopting the open-source `simpleuart` core resolved this. Preserving a fully working, deployable design at the end of every stage, rather than accumulating partially integrated features, was the practice that kept the overall schedule recoverable despite these setbacks.

The overarching lesson is that hardware project timelines are dominated by integration and debug rather than initial design. The Gantt charts in Appendix B record both the original plan and the way the work actually unfolded.

Chapter 8

Conclusion

8.1 Summary of Achievements

This project set out to design, implement, and evaluate a RISC-V-attached hardware accelerator for CSR sparse matrix–dense matrix multiplication on the Terasic DE1-SoC, and to demonstrate a measurable speedup over software while remaining reproducible at the register-transfer level. That aim was met. A complete PicoRV32 SoC with a working SpMM accelerator, 64 KB shared dual-port BRAM, MMIO interface, and UART reporting was synthesised and deployed; three recorded optimisation stages (baseline, DSP-enhanced, and INT8 packed) were each verified for correctness on hardware; and a 12-case benchmark harness with deterministic matrix generation, cycle-accurate timing, and element-by-element correctness checking was developed to evaluate them.

The final INT8-packed implementation achieves a peak speedup of **56.80**× over the quantised PicoRV32 software reference and **11.92**× over an ARM Cortex-M0 baseline running the same algorithm, with an average of 45.28× and 9.79× respectively across the 12-case suite. All 12 cases passed correctness checks at every stage.

8.2 Significance

The significance of this work lies not in the absolute numbers compared to large research accelerators, but in what the staged measurement reveals about the

workload. The 1.32× gain from DSP-based MAC versus the 2.18× gain from INT8 packing experimentally confirms that the dominant bottleneck in CSR SpMM on a memory-bandwidth-constrained embedded FPGA is operand fetch traffic, not arithmetic execution. This is a useful result beyond the specific artefact: it indicates that similar embedded accelerators should prioritise format-level and precision-level optimisations over wider MAC arrays. The project also demonstrates that a fully integrated, firmware- driven hardware benchmark harness is a practical alternative to external simulation infrastructure for small-scale accelerator evaluation, and that a useful SpMM accelerator fits comfortably within the resource budget of a Cyclone V device.

8.3 Future Work

The most impactful extensions to the current design would be:

1. **Wider tile width.** INT8 packing reduces the per-nonzero fetch cost enough that widening from $TN = 8$ to $TN = 16$ or $TN = 32$ is now attractive. Doubling the tile width costs an additional two 32-bit reads per nonzero but delivers 2× the MAC throughput per nonzero, and should push the speedup ceiling further.

2. **Ping-pong B-segment prefetch.** Double-buffering the B-segment allows the next segment to be fetched while the current MAC operation completes, overlapping memory latency with arithmetic and removing it from the critical path.

3. **External memory interface.** Extending the design to address off-chip SDRAM or HPS DRAM through the DE1-SoC's HPS bridge would permit realistically sized GNN and recommender-system matrices to be benchmarked, at which point off-chip bandwidth becomes the dominant term and different optimisations become relevant.

4. **Interrupt-driven completion.** Replacing the polling loop with a RISC-V interrupt handler would free the CPU during accelerator execution, enabling pipelined benchmark preparation or post-processing of the previous result.

5. **Power and energy characterisation.** Instrumenting the DE1-SoC rails or using Quartus Power Analyser with simulation-derived toggle files would

add energy efficiency to the evaluation, which is the metric that matters most in the embedded and edge contexts for which this class of accelerator is targeted.

8.4 Closing Remarks

This project has delivered a complete, measured, and reproducible RISC-V FPGA-attached SpMM accelerator that achieves a peak speedup of 56.80× on hardware through three progressively optimised designs. The work confirms that even without exotic hardware resources or advanced scheduling, careful alignment of the memory format, datapath structure, and quantisation strategy with the actual bottleneck of the workload produces large, predictable, and experimentally confirmed performance gains.

Bibliography

[1] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30. Curran Associates, 2017.

[2] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[3] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for YouTube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*, pages 191–198. ACM, 2016. doi: 10.1145/2959100.2959190.

[4] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. EIE: Efficient inference engine on compressed deep neural network. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, pages 243–254. IEEE, 2016. doi: 10.1109/ISCA.2016.30.

[5] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

[6] Fred G. Gustavson. Two fast algorithms for sparse matrices: Multiplication and permuted transposition. *ACM Transactions on Mathematical Software*, 4 (3):250–269, 1978. doi: 10.1145/355791.355796.

[7] Nathan Bell and Michael Garland. Implementing sparse matrix-vector multiplication on throughput-oriented processors. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis (SC'09)*, pages 1–11. ACM, 2009. doi: 10.1145/1654059.1654078.

[8] Subhankar Pal, Jonathan Beaumont, Dong-Hyeon Park, Aporva Amarnath, Siying Feng, Chaitali Chakrabarti, Hun-Seok Kim, David Blaauw, Trevor Mudge, and Ronald Dreslinski. OuterSPACE: An outer product based sparse matrix multiplication accelerator. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 724–736. IEEE, 2018. doi: 10.1109/HPCA.2018.00067.

[9] Zhekai Zhang, Hanrui Wang, Song Han, and William J. Dally. SpArch: Efficient architecture for sparse matrix multiplication. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 261–274. IEEE, 2020. doi: 10.1109/HPCA47549.2020.00030.

[10] Kartik Hegde, Po-An Tsai, Sheng-Chun Huang, Vikas Chandra, Angshuman Parashar, and Christopher W. Fletcher. ExTensor: An accelerator for sparse tensor algebra. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 319–333. ACM, 2019. doi: 10.1145/3352460.3358275.

[11] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. SIGMA: A sparse and irregular GEMM accelerator with flexible interconnects for DNN training. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 58–70. IEEE, 2020. doi: 10.1109/HPCA47549.2020.00015.

[12] Eriko Nurvitadhi, Ganesh Venkatesh, Jaewoong Sim, Debbie Marr, Randy Huang, Jason Ong Gee Hock, Yeong Tat Liew, Krishnan Srivatsan, Duncan Moss, Suchit Subhaschandra, and Guy Ruhl. Can FPGAs beat GPUs in accelerating next-generation deep neural networks? In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, pages 5–14. ACM, 2017. doi: 10.1145/3020078.3021740.

[13] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2704–2713. IEEE, 2018. doi: 10.1109/CVPR.2018.00286.

[14] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021.

[15] Terasic Technologies. DE1-SoC User Manual. https://courses.cs.washington.edu/courses/cse371/references/DE1-SoC_User_Manual.pdf, 2017. Accessed: 2026-03-03.

[16] Claire Wolf and contributors. PicoRV32: A Size-Optimized RISC-V CPU. https://github.com/YosysHQ/picorv32, 2026. GitHub repository, Accessed: 2026-03-03.

[17] RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. https://riscv.github.io/riscv-isa-manual/snapshot/unprivileged/, 2024. Accessed: 2026-03-03.

[18] Intel Corporation. Cyclone V Device Handbook. https://www.intel.com/content/www/us/en/docs/programmable/683375/current/cyclone-v-device-overview.html, 2024. Accessed: 2026-03-03.

[19] Intel Corporation. Cyclone V Device: Embedded Memory User Guide. https://www.intel.com/content/www/us/en/docs/programmable/683179/current/, 2022. Accessed: 2026-03-03.

[20] Intel Corporation. Cyclone V Device: DSP Blocks User Guide. https://www.intel.com/content/www/us/en/docs/programmable/683364/current/, 2022. Accessed: 2026-03-03.

[21] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. Efficient processing of deep neural networks: A tutorial and survey. *Proceedings of the IEEE*, 105(12):2295–2329, 2017. doi: 10.1109/JPROC.2017.2761740.

Appendix A

CSR Encoding and Quantisation Reference

This appendix is a technical reference for the two data representations used throughout the project. It complements Chapter 2 by supplying explicit worked examples, a storage-cost derivation, row-wise SpMM pseudocode, and a numeric quantisation example, all of which would interrupt the flow of the main chapter if included inline.

A.1 CSR Format Definition

A sparse matrix A of size $M \times K$ with NNZ nonzeros is stored in compressed sparse row (CSR) as three arrays:

- `rowptr[0..M]`: an array of $M + 1$ integers, where `rowptr[i]` is the index into `colidx` and `values` at which row i begins. The sentinel `rowptr[M] = NNZ` closes the final row.
- `colidx[0..NNZ-1]`: the column index of each nonzero, listed in row-major order.
- `values[0..NNZ-1]`: the nonzero scalar values, aligned positionally with `colidx`.

The nonzeros of row i occupy indices `rowptr[i]` through `rowptr[i+1] - 1` in both `colidx` and `values`, so the row length is `rowptr[i+1] - rowptr[i]`. An empty row i is represented by `rowptr[i+1]` = `rowptr[i]`.

A.2 Worked CSR Encoding Example

Consider the 4×5 sparse matrix

$$A = \begin{pmatrix} 0 & 4 & 0 & 0 & 2 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 6 & 3 & 5 \end{pmatrix},$$

which has $\mathrm{NNZ} = 6$ and includes an empty row (row 2) and a dense trailing row (row 3). Its CSR encoding is

$$\begin{aligned} \texttt{rowptr} &= [0,\ 2,\ 3,\ 3,\ 6] \\ \texttt{colidx} &= [1,\ 4,\ 0,\ 2,\ 3,\ 4] \\ \texttt{values} &= [4,\ 2,\ 1,\ 6,\ 3,\ 5] \end{aligned}$$

Row 2 is recovered as `rowptr[3] - rowptr[2]` $= 3 - 3 = 0$ nonzeros, so the iteration below produces no updates for that row. This example differs from the Chapter 2 illustration deliberately: the empty row exercises the pointer-difference logic that real sparse workloads exhibit.

A.3 Storage Cost Analysis

Dense storage of an $M \times K$ matrix requires $M \cdot K$ elements. CSR requires $(M + 1) + 2 \cdot \mathrm{NNZ}$ elements split across the three arrays. Writing the density as $\rho = \mathrm{NNZ}/(M \cdot K)$, CSR is more compact than dense storage whenever

$$(M + 1) + 2\rho MK \ < \ MK \quad \Longleftrightarrow \quad \rho \ < \ \tfrac{1}{2}\left(1 - \tfrac{M+1}{MK}\right).$$

For $M = K = 64$, this break-even density is approximately 49.2%, i.e. CSR wins for any sparsity above roughly 50%. In this project the benchmark densities lie in the range 2%–25%, so CSR is always substantially more compact than dense storage.

Assuming 32-bit words for every field (the layout used in firmware), the sparse matrix of Section A.2 occupies $(4 + 1) + 2 \cdot 6 = 17$ words, versus $4 \cdot 5 = 20$ words for the dense representation. Under the INT8 packed encoding used in the final stage, `values` shrinks to $\lceil \mathrm{NNZ}/4 \rceil$ words while `rowptr` and `colidx` remain 32-bit; total storage for the example falls to $(4 + 1) + 6 + 2 = 13$ words.

A.4 Row-wise CSR SpMM Pseudocode

The row-wise SpMM algorithm used by both the software baseline and the accelerator is:

```
Input:  rowptr[0..M], colidx[0..NNZ-1], values[0..NNZ-1],
        B[K x N], N
Output: C[M x N]

for i in 0..M-1:                        #iterate rows of A
    for j in 0..N-1:                    #clear row i of C
        C[i][j] = 0
    for p in rowptr[i]..rowptr[i+1]-1: #nonzeros of row i
        a = values[p]
        k = colidx[p]
        for j in 0..N-1:                #accumulate a * B[k, :]
            C[i][j] += a * B[k][j]
```

Two properties matter for hardware mapping. First, row i of C is independent of every other row, so the outer loop is embarrassingly parallel and amenable to tiling across the dense dimension N. Second, the inner j-loop touches a contiguous run of $B[k, :]$, which streams cleanly from block RAM and is the access pattern the accelerator exploits through its $TN = 8$ parallel MAC lanes.

A.5 Why CSR over COO, ELL, and BSR

Chapter 2 compares the main sparse formats at a high level; the specific reasons CSR was chosen for this accelerator are:

- **Constant-time row access.** Reading `rowptr[i]` and `rowptr[i+1]` gives the row extents in two loads, which maps directly to two BRAM reads by the control FSM. COO requires a linear scan for the row boundary, so row access would take $O(\text{NNZ})$ in the worst case.
- **No padding or wasted storage.** ELL pads every row to the length of the densest row, which wastes storage on matrices with irregular row lengths (for example the power-law sparsity case in the benchmark suite). CSR stores only the actual nonzeros.

- **Streaming-friendly layout.** `colidx` and `values` are traversed in lock-step, so a single address counter drives both BRAMs; BSR would require block-level addressing and a small dense multiplier per block, which is an order of magnitude more hardware for little gain at the sparsity levels targeted here.

A.6 Symmetric Quantisation and Dequantisation

Let $\mathbf{v} \in \mathbb{R}^n$ be a real-valued vector. Define $v_{\max} = \max_i |v_i|$. Symmetric INT8 quantisation is the pair of functions

$$\text{quantise}(v_i; s) \;=\; \text{clip}\big(\text{round}\big(\tfrac{v_i}{s}\big),\; -127,\; 127\big)\,, \qquad \text{dequantise}(q_i; s) \;=\; q_i \cdot s,$$

where the scale factor $s = v_{\max}/127$ maps the largest absolute input to the largest representable magnitude. The clip to $[-127, 127]$ rather than $[-128, 127]$ is intentional: reserving -128 preserves the sign symmetry $\text{quantise}(-v_i) = -\text{quantise}(v_i)$ exactly, at the cost of losing one codepoint. Because the mapping is linear and origin-preserving, $\text{quantise}(0; s) = 0$ for any $s > 0$.

A.7 Edge Case: $v_{\max} = 0$

If every element of $\mathbf{v}$ is zero then $v_{\max} = 0$ and the scale factor is undefined. The firmware handles this case explicitly: it detects $v_{\max} = 0$ before the division, sets $s = 1$ by convention, and emits an all-zero quantised vector. The downstream accelerator then multiplies zero by whatever it reads, so the output is zero regardless of the scale choice. Omitting this guard would trigger a division by zero in the scale computation and corrupt the quantised stream.

A.8 Why Symmetric Avoids a Zero-Point Offset

A general affine quantiser takes the form $q_i = \text{round}(v_i/s) + z$, where z is a zero-point offset chosen so that $v_i = 0$ maps to a specific integer codepoint. Reconstruction is then $\hat{v}_i = (q_i - z) \cdot s$, and accumulation over a dot product $\sum_i a_i b_i$ becomes

$$\sum_i (q_i^a - z_a)(q_i^b - z_b) \cdot s_a s_b \;=\; s_a s_b \sum_i \left(q_i^a q_i^b - z_a q_i^b - z_b q_i^a + z_a z_b\right),$$

which introduces three cross-terms that have to be computed or precomputed and subtracted before the final scale multiplication. Symmetric quantisation fixes $z_a = z_b = 0$, so the expansion collapses to $\sum_i q_i^a q_i^b \cdot s_a s_b$; the hardware MAC array sees a clean signed multiply-accumulate with no pre- or post-subtract stage. This directly saved an adder and a multiplier per lane in the INT8 stage of the datapath.

A.9 Numeric Quantisation Example

Take the real-valued vector

$$\mathbf{v} = (0.40,\ -1.20,\ 2.80,\ -0.70,\ 1.90).$$

Then $v_{\max} = 2.80$ and $s = 2.80/127 \approx 0.022047$. Applying the quantise and dequantise functions of Section A.6 element by element gives the results in Table A.1.

i	v_i	q_i	$\hat{v}_i$	$\lvert e_i\rvert = \lvert v_i - \hat{v}_i\rvert$
0	+0.40	+18	+0.3969	0.0031
1	−1.20	−54	−1.1905	0.0095
2	+2.80	+127	+2.8000	0.0000
3	−0.70	−32	−0.7055	0.0055
4	+1.90	+86	+1.8960	0.0040

TABLE A.1: Worked symmetric INT8 quantisation of $\mathbf{v} = (0.40, -1.20, 2.80, -0.70, 1.90)$ with scale $s = 2.80/127 \approx 0.022047$. The maximum observed absolute error is 0.0095, below the theoretical bound $s/2 \approx 0.01102$.

Three observations follow. The largest-magnitude element ($v_2 = 2.80$) maps exactly to the boundary codepoint +127 and incurs zero reconstruction error, which is the defining property of symmetric quantisation at the maximum. The signs of all elements are preserved, confirming that the origin-preserving property of the mapping carries through the round-to-nearest step. Finally, every observed $|e_i|$ lies below $s/2$, consistent with the general bound $|e_i| \leq s/2 = v_{\max}/254$ from rounding to the nearest integer multiple of s.

Appendix B

Gantt Charts and Project Plan

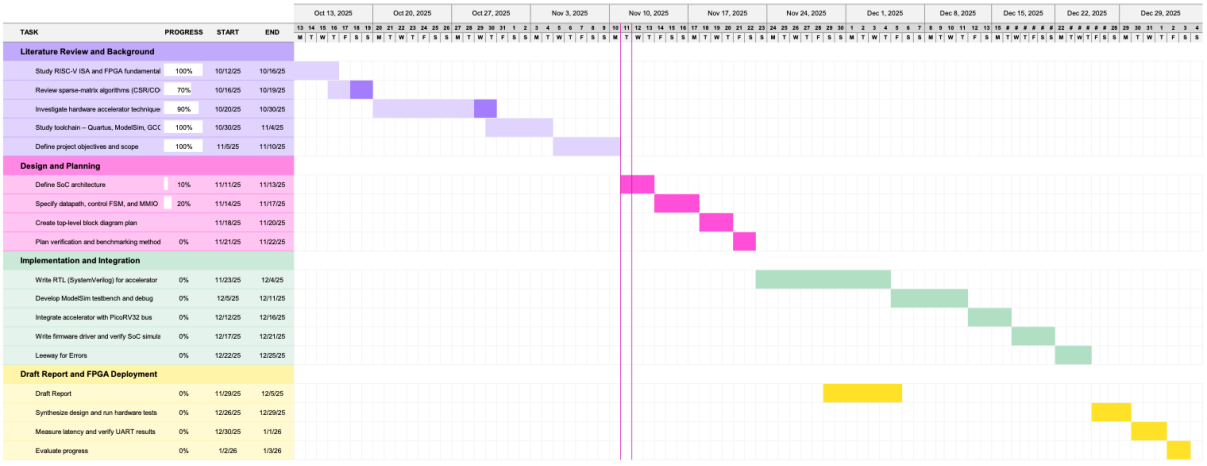

FIGURE B.1: Planned tasks for the first semester.

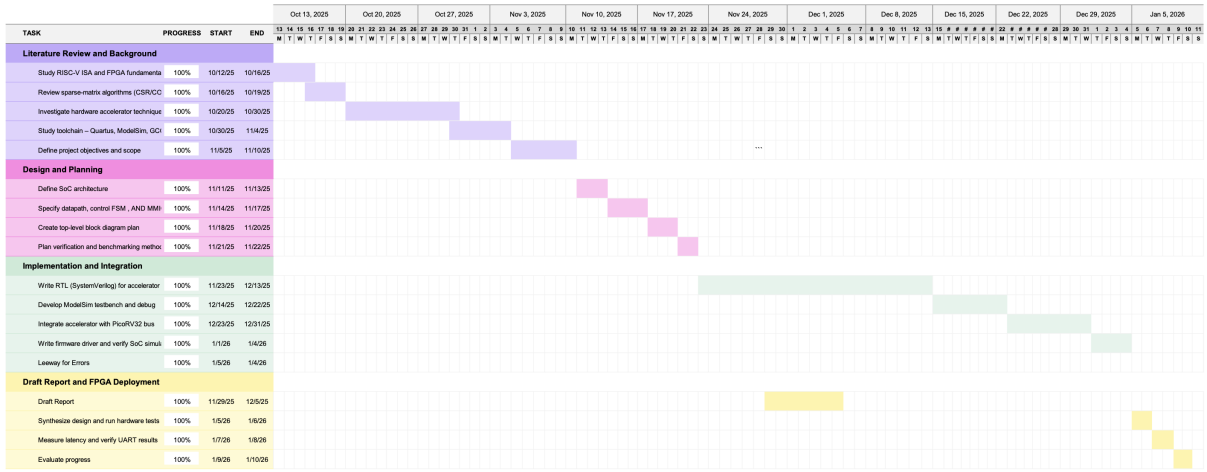

FIGURE B.2: Completed Gantt chart for the first semester.

At the time of the interim report submission, the implementation and integration phases experienced several delays. Consequently, the implementation phase took two weeks longer than originally anticipated. Conversely, the initial FPGA deployment was completed just before the Semester 1 examination period and required less time than budgeted, as illustrated in Figure B.2.

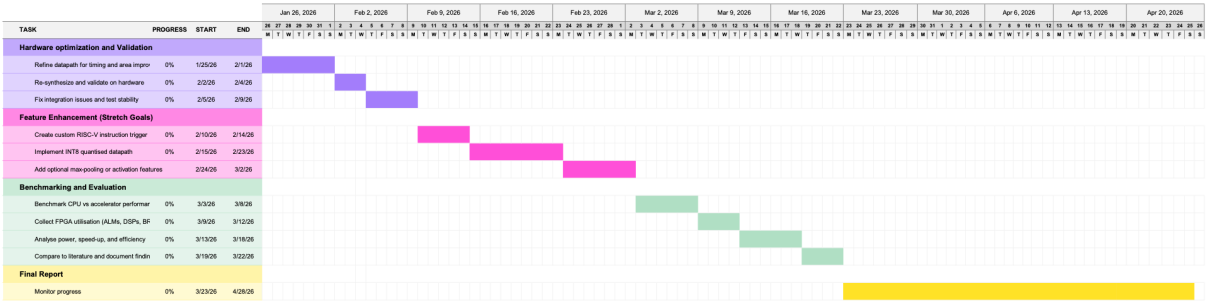

FIGURE B.3: Planned project schedule for Semester 2, covering optimization stages, firmware benchmark harness, FPGA deployment, and report writing.

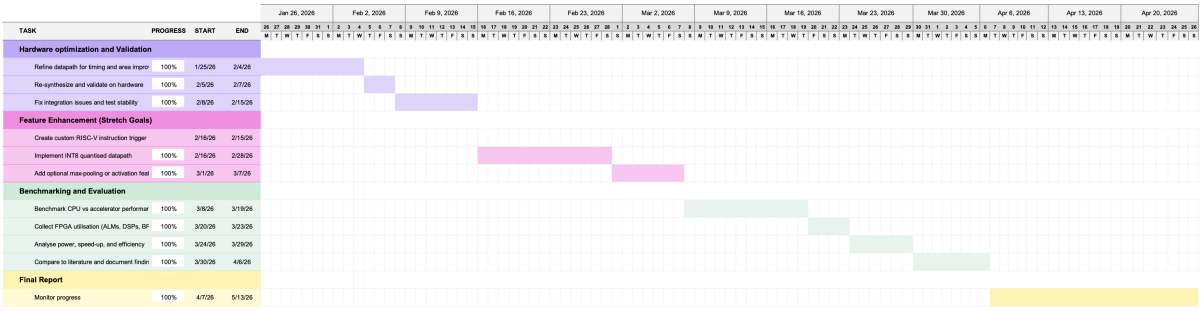

FIGURE B.4: Actual effort distribution for Semester 2, showing the impact of integration and timing closure delays.

The original plan for Semester 2 (Figure B.3) allocated a 32-day window for report writing, spanning approximately from March 23rd to April 25th. However, as illustrated in Figure B.4, the benchmarking and evaluation phases significantly exceeded their projected timelines. This delay necessitated a substantial reduction in the time available for finalizing the report. Consequently, the stretch goal involving custom RISC-V instructions for Sparse Matrix-Vector Multiplication (SpMM) was deprioritized; attempting this implementation would have jeopardized the completion of more critical project milestones.

Appendix C

Design and Data Archive

The complete project repository is organised as follows:

- `rtl/`: accelerator RTL, comprising the top-level wrapper (`accel_top.sv`), the control finite state machine (`accel_ctrl.sv`), and the parallel MAC datapath (`accel_datapath.sv`).
- `fpga/rtl/`: synthesisable SoC top-level (`soc_top.sv`) instantiating PicoRV32, the dual-port BRAM, the accelerator, the UART peripheral, and the GPIO interface.
- `fpga/quartus/`: Quartus project files (`de1_soc_spmm.qpf`), pin assignment constraints (`de1_soc_spmm.qsf`), timing constraints (`de1_soc_spmm.sdc`), and the byte-lane memory initialisation files (`firmware[0-3].mif`) generated from the compiled firmware image.
- `sw/driver/`: bare-metal firmware, including the RISC-V assembly startup (`start.S`), benchmark harness (`main.c`), accelerator driver (`spmm_accel.c/.h`), libc stubs (`libc_stubs.c`), linker script (`linker.ld`), `Makefile`, and the Python helper scripts (`bin2hex32.py`, `hex2mif.py`) that convert the compiled ELF into the byte-lane MIF files consumed by Quartus.
- `tb/common/`: shared testbench infrastructure, comprising the clock/reset generator (`clk_reset.sv`), the MMIO SystemVerilog interface (`mmio_if.sv`), the shared utility package (`tb_pkg.sv`), the assertion macros (`tb_macros.svh`), and the shared MMIO register-map header (`tb_regmap.svh`).
- `tb/unit/`: unit testbenches for the individual accelerator modules (`accel_ctrl_tb.sv`, `accel_datapath_tb.sv`, `accel_top_tb.sv`).

- `tb/integ/`: integration testbench (`soc_tb.sv`) that exercises the full synthesisable SoC.
- `sim/`: simulator command scripts (`compile_soc.do`) for invoking the testbenches under ModelSim/Questa.
- `ip/`: third-party IP cores used unmodified, namely the PicoRV32 RV32IM soft core (`ip/picorv32/`) and the Cortex-M0 reference source (`ip/arm_m0/`) used to derive the Cortex-M0 cycle estimates reported in Table 6.6.
- `docs/my_report/`: report LaTeX source, bibliography (`bib/ECS.bib`), figure assets, and chapter files under `chapters/`.
- `README.md`: top-level repository overview, build instructions, and quick-start guide for reproducing the benchmark results.

All source, testbench, firmware, and Quartus project files required to reproduce the measurements reported in Chapter 6 are contained within this repository.